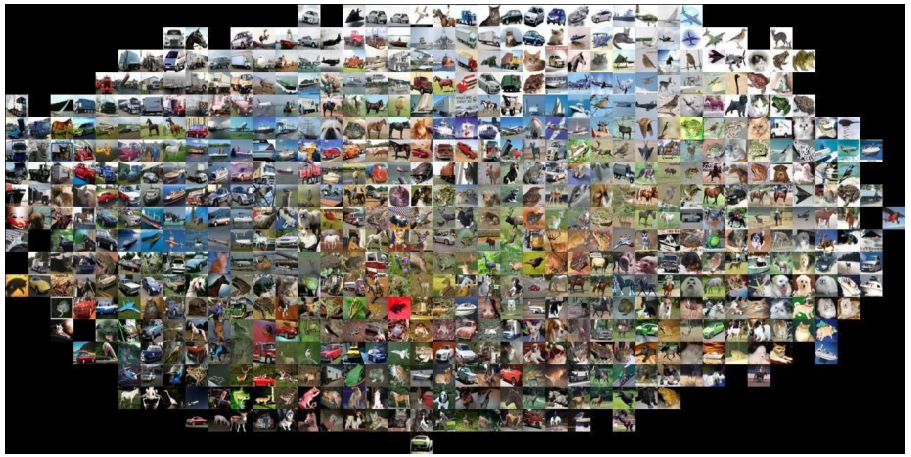
BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Computer Vision

S1-24_AIMLCZG525, M.Tech (AIML)



Session # 9:

Image Classification

Dhruba Adhikary

BITS Pilani
Pilani | Dubai | Goa | Hyderabad

Topics

- ➔ Image classification pipeline
- ➔ Training, validation, testing
- ➔ Nearest neighbor classification
- ➔ Linear classification
- ➔ Building up to… CNNs for learning, transformers and vision transformers (next class)

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely, Fei-Fei Li, Justin Johnson, Serena Yeung

What matters in recognition?

- Machine learning methods (e.g., linear classification, deep learning)
- Representation (e.g., SIFT, HoG, deep learned features)
- **Data** (e.g., PASCAL, ImageNet, COCO)

Image Classification:
A core task in Computer Vision

- Assume given set of discrete labels, e.g.

{cat, dog, cow, apple, tomato, truck, … }

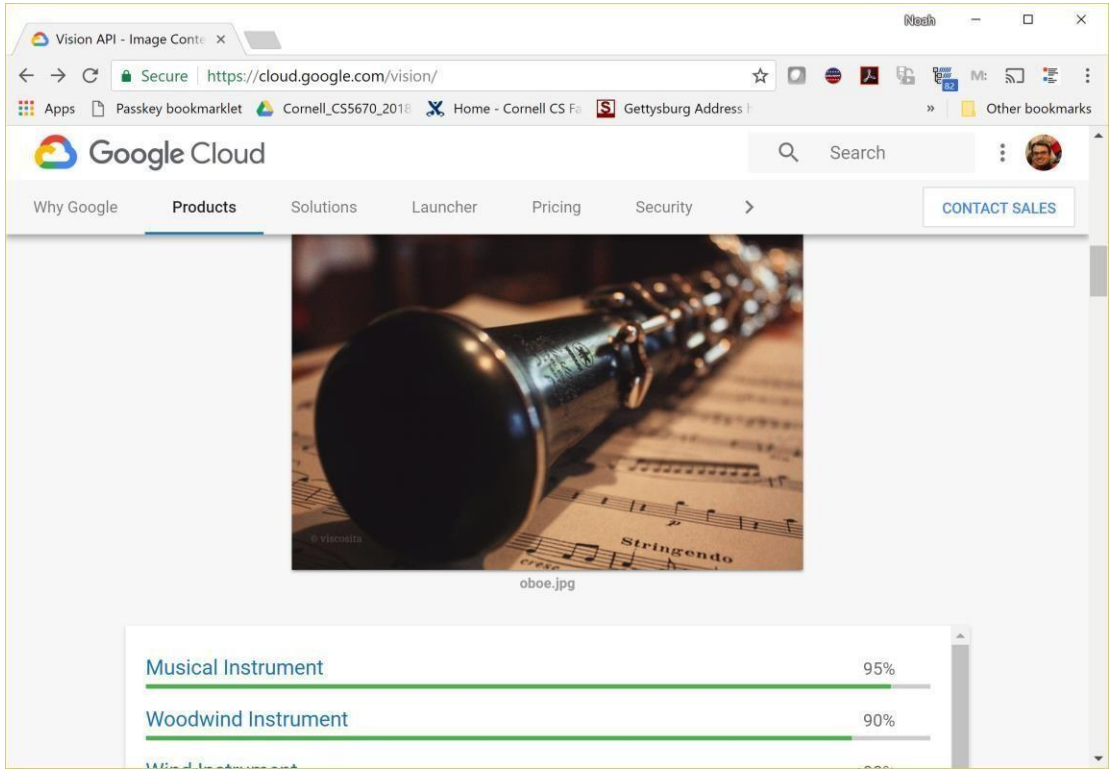
f() = “apple”

f() = “tomato”

f() = “cow”

Dataset: ETH-80, by B. Leibe Slide credit: L. Lazebnik

Image classification demo



https://cloud.google.com/vision/

See also: https://aws.amazon.com/rekognition/
https://www.clarifai.com/
https://azure.microsoft.com/en-us/services/cognitive-services/computer-vision/

…

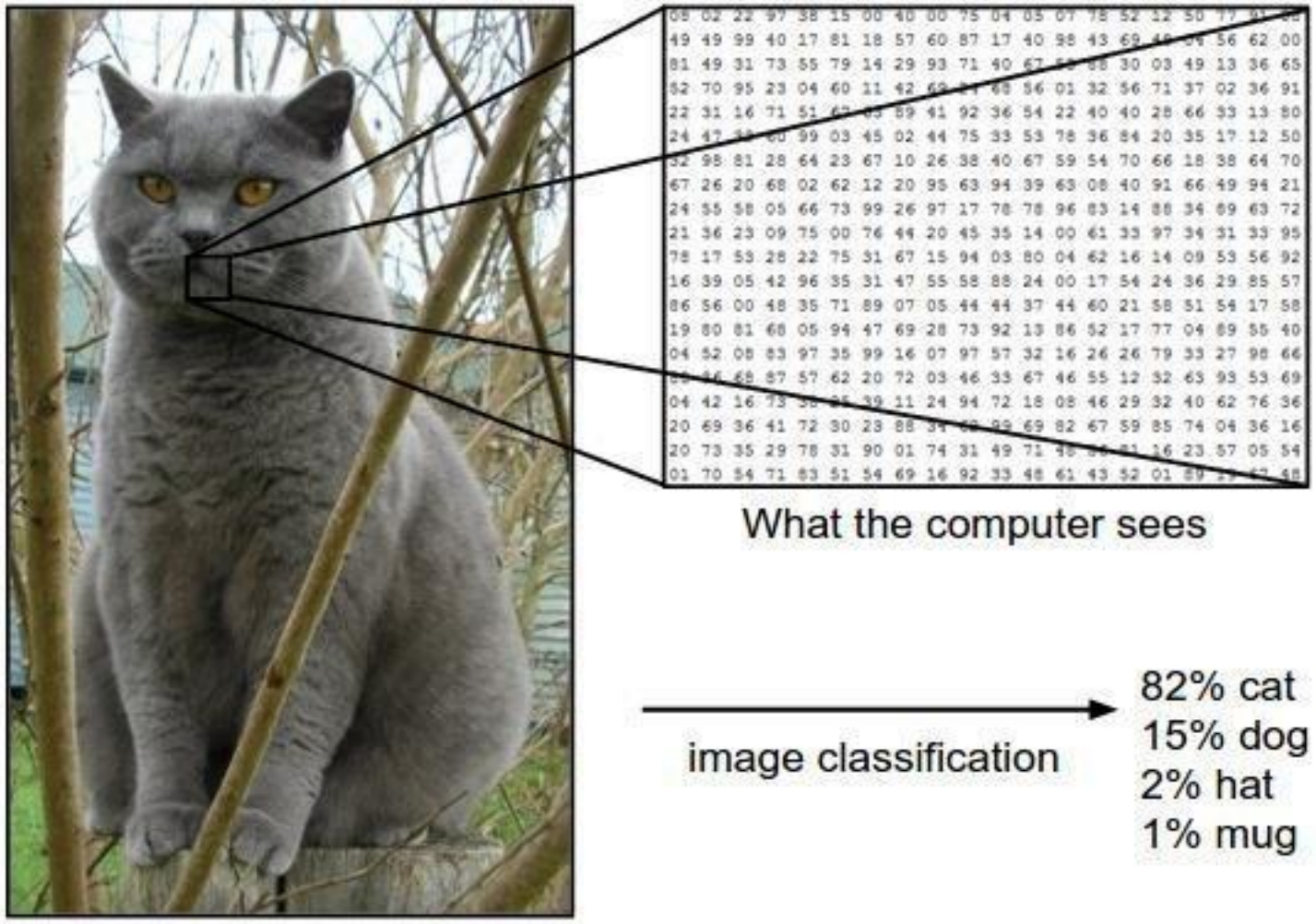
Image Classification

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

cat

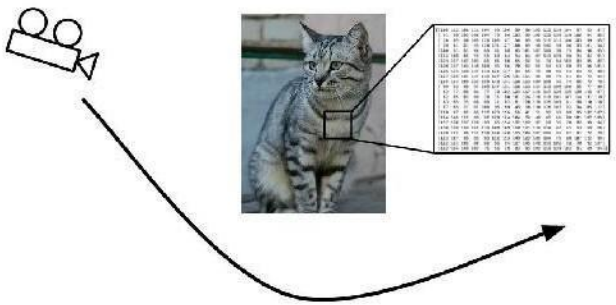
Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

Image Classification: Problem



Recall from last time: Challenges of recognition

Viewpoint



Illumination

This image is CC0 1.0 public domain

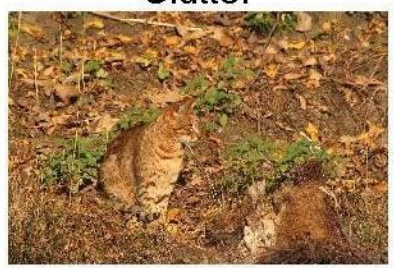
Deformation

This image by Umberto Salvagnin is licensed under CC-BY 2.0

Occlusion

This image by jonsson is licensed under CC-BY 2.0

Clutter



This image is CC0 1.0 public domain

Intraclass Variation

This image is CC0 1.0 public domain

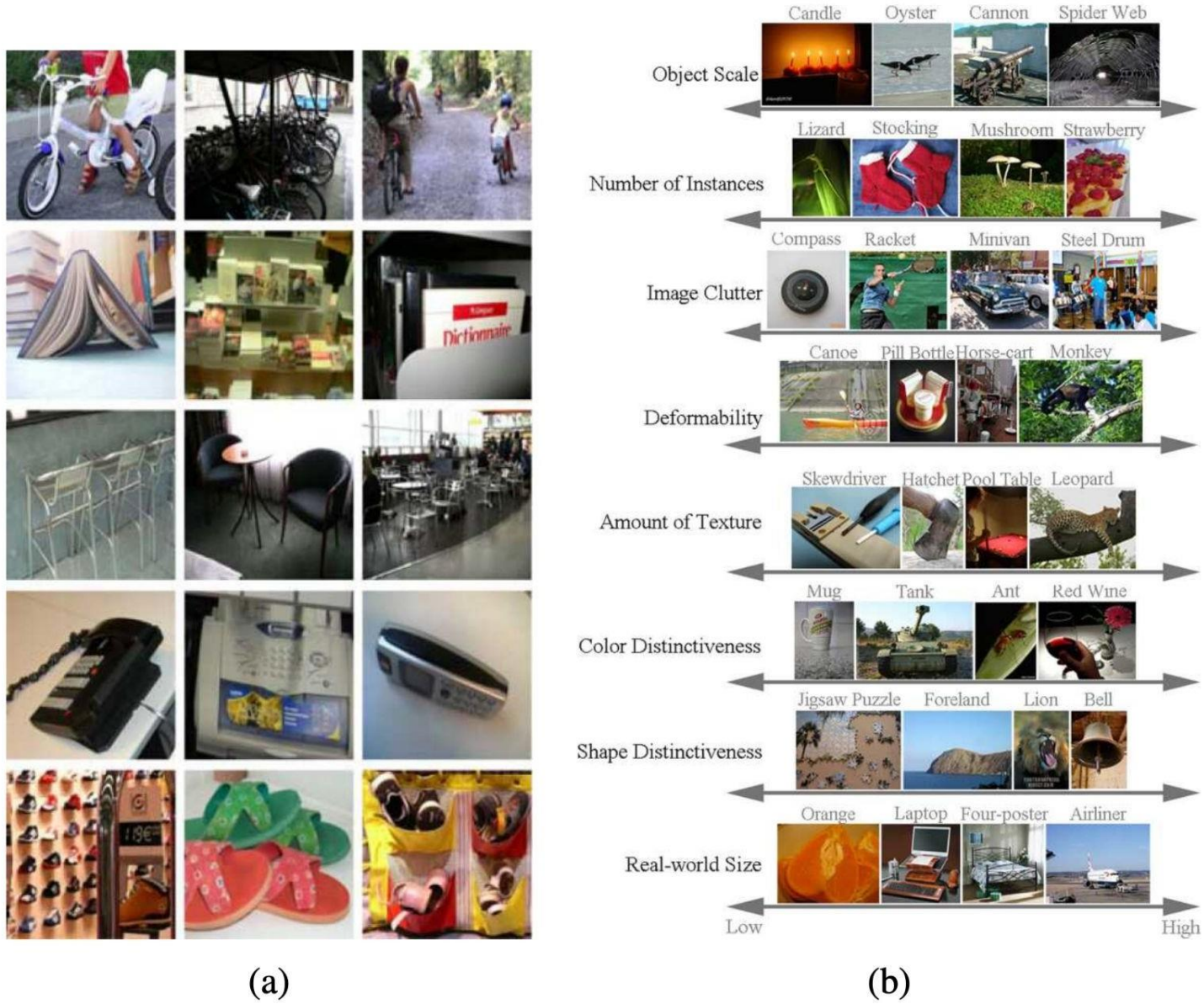


Figure 6.4 *Challenges in image recognition: (a) sample images from the Xerox 10 class dataset (Csurka, Dance* et al. *2006) © 2007 Springer; (b) axes of difficulty and variation from the ImageNet dataset (Russakovsky, Deng* et al. *2015) © 2015 Springer.*

An image classifier

```
def classify_image(image):
  # Some magic here?
  return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

Slide credit: Fei-Fei Li & Justin Johnson & Serena Yeung

Data-driven approach

- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images

Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

Data-driven approach

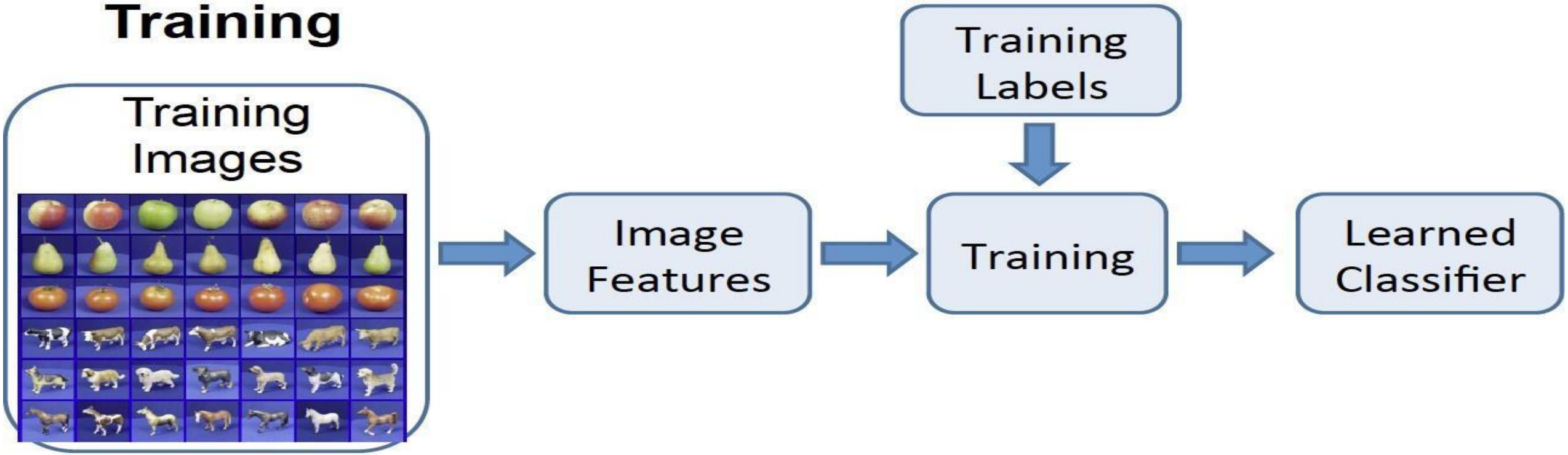
- Collect a database of images with labels
- Use ML to train an image classifier
- Evaluate the classifier on test images

```
def train(train_images, train_labels):
    # build a model of images -> labels

def predict(image):
    # evaluate the model on the image
    return class_label
```

Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

Training



Dataset: ETH-80, by B. Leibe Slide credit: D. Hoiem, L. Lazebnik

Training

Training Images

Training Labels

Image Features

Training

Learned Classifier

Testing

Test Image

Image Features

Learned Classifier

Prediction

Dataset: ETH-80, by B. Leibe Slide credit: D. Hoiem, L. Lazebnik

Bag of Words

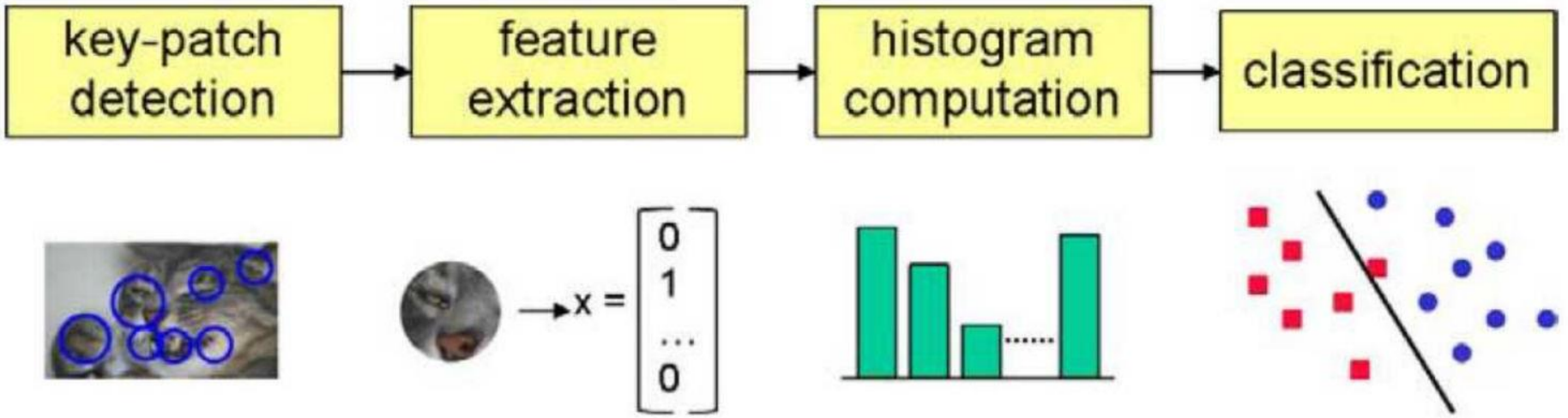


Figure 6.6 *A typical processing pipeline for a bag-of-words category recognition system (Csurka, Dance* et al. *2006) © 2007 Springer. Features are first extracted at keypoints and then quantized to get a distribution (histogram) over the learned* visual words *(feature cluster centers). The feature distribution histogram is used to learn a decision surface using a classification algorithm, such as a support vector machine.*

Computer Vision: Algorithms and Applications, 2nd ed.

Classifiers

- Nearest Neighbor
- kNN (“k-Nearest Neighbors”)
- Linear Classifier
- Neural Network
- Deep Neural Network
- …

First: Nearest Neighbor (NN) Classifier

- Train
  - Remember all training images and their labels
- Predict
  - Find the closest (most similar) training image
  - Predict its label as the true label

CIFAR-10 and NN results

Example dataset: **CIFAR-10**
10 labels
50,000 training images, each image is tiny: 32x32
10,000 test images.

CIFAR-10 and NN results

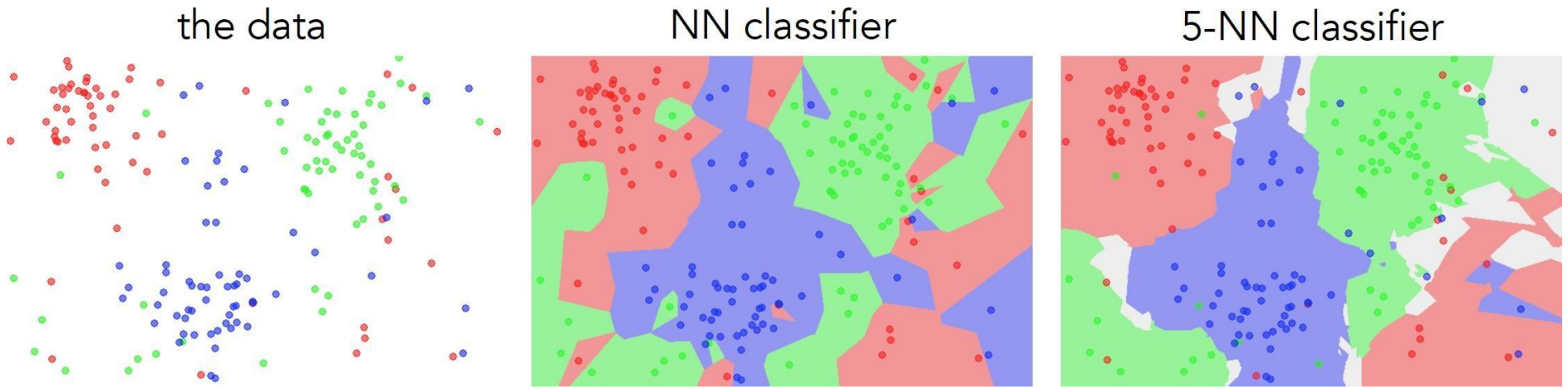
Example dataset: **CIFAR-10**
10 labels
50,000 training images
10,000 test images.

For every test image (first column), examples of nearest neighbors in rows

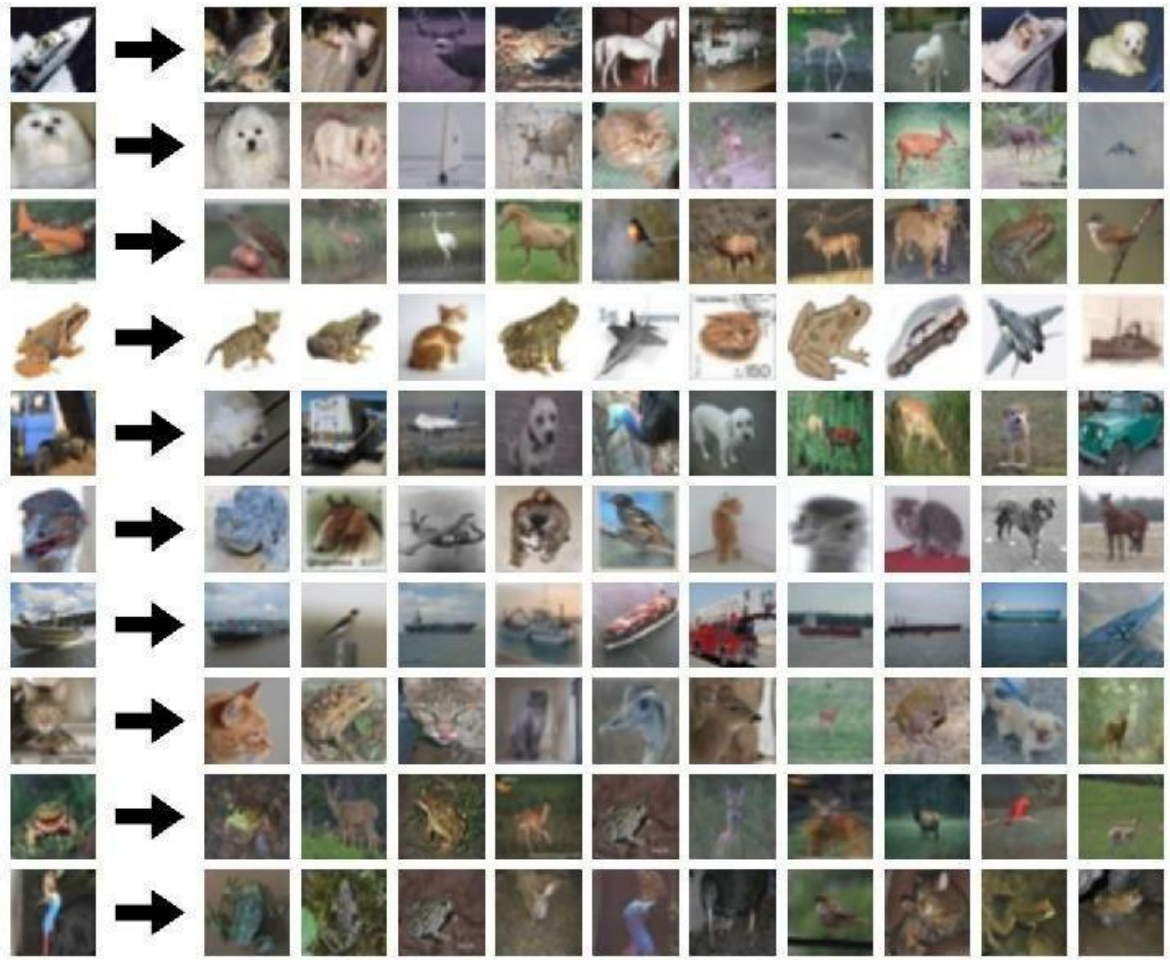
Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

k-nearest neighbor

- Find the k closest points from training data
- Take **majority vote** from K closest points



What does this look like?



What does this look like?

How to find the most similar training image? What is the distance metric?

L1 distance: $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$

Where I_1 denotes image 1, and p denotes each pixel

test image					training image					pixel-wise absolute value differences				
56	32	10	18	-	10	20	24	17	=	46	12	14	1	→ 456
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200		12	16	178	170		12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	

Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

Choice of distance metric

- Hyperparameter

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$

L2 (Euclidean) distance

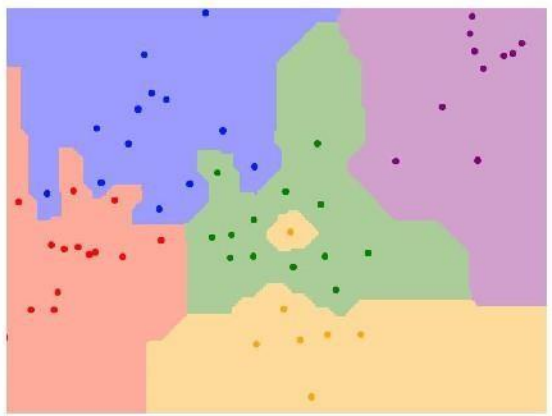
$$d_2(I_1, I_2) = \sqrt{\sum_p \left(I_1^p - I_2^p\right)^2}$$

Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

K-Nearest Neighbors: Distance Metric

L1 (Manhattan) distance

$$d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$$



K = 1

L2 (Euclidean) distance

$$d_2(I_1, I_2) = \sqrt{\sum_p (I_1^p - I_2^p)^2}$$

K = 1

Demo: http://vision.stanford.edu/teaching/cs231n-demos/knn/

Hyperparameters

- What is the **best distance** to use?
- What is the **best value of k** to use?

- These are **hyperparameters**: choices about the algorithm that we set rather than learn

- How do we set them?
  - One option: try them all and see what works best

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

BAD: No idea how algorithm will perform on new data

Setting Hyperparameters

Idea #1: Choose hyperparameters that work best on the data

BAD: K = 1 always works perfectly on training data

Idea #2: Split data into **train** and **test**, choose hyperparameters that work best on test data

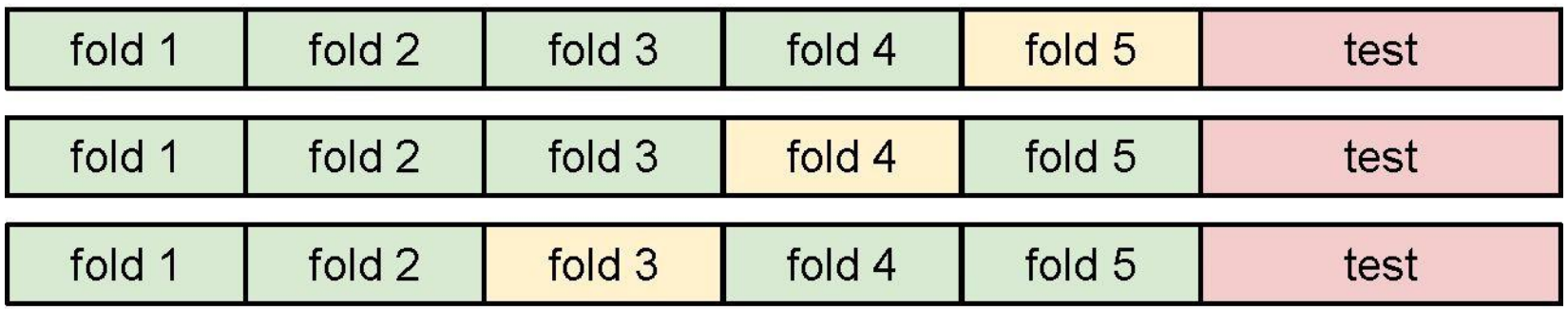
BAD: No idea how algorithm will perform on new data

Idea #3: Split data into **train**, **val**, and **test**; choose hyperparameters on val and evaluate on test

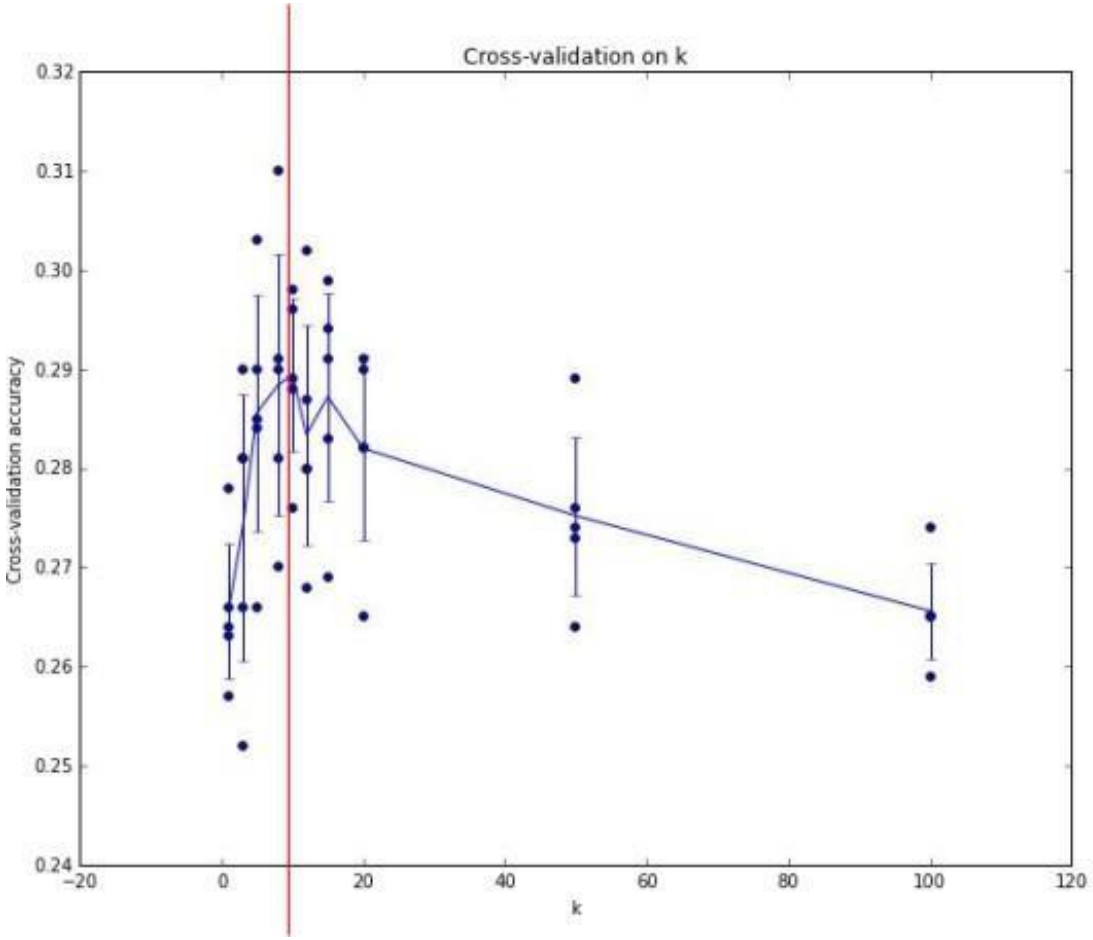
Better!

Setting Hyperparameters

Idea #4: **Cross-Validation**: Split data into **folds**,
try each fold as validation and average the results



Useful for small datasets, but not used too frequently in deep learning



Example of
5-fold cross-validation
for the value of **k.**

Each point: single
outcome.

The line goes
through the mean, bars
indicated standard
deviation

(Seems that k ~= 7 works best
for this data)

Recap: How to pick hyperparameters?

- Methodology
  - Train and test
  - Train, validate, test

- Train for original model
- Validate to find hyperparameters
- Test to understand generalizability

kNN -- Complexity and Storage

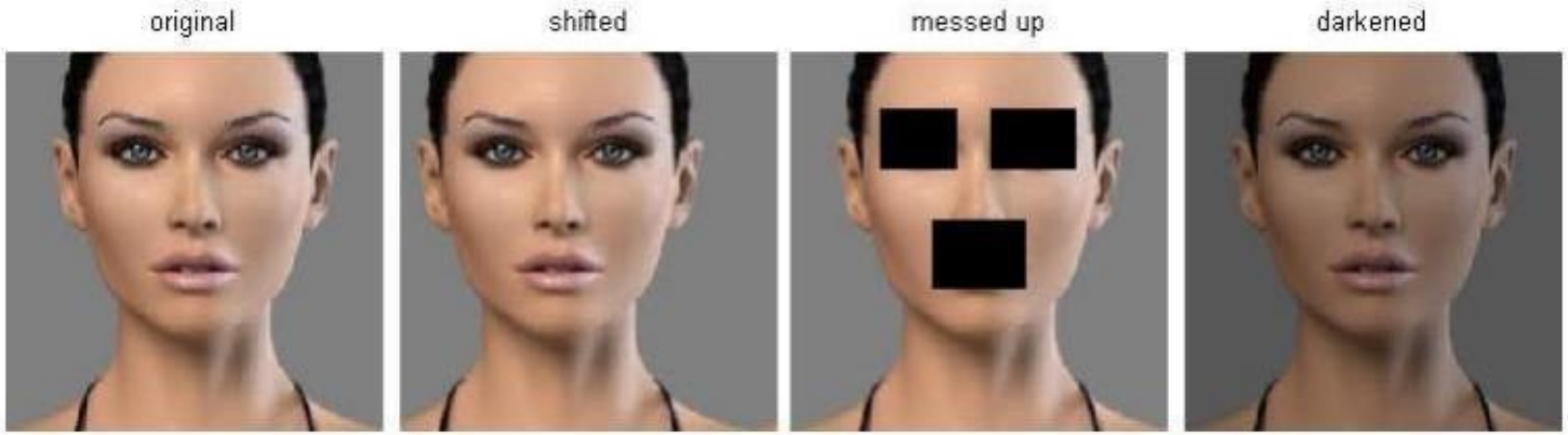
- N training images, M test images

- Training: O(1)
- Testing: O(MN)

- Hmm…
  - Normally need the opposite
  - Slow training (ok), fast testing (necessary)

k-Nearest Neighbor on images **never used.**

- terrible performance at test time
- distance metrics on level of whole images can be very unintuitive



(all 3 images have same L2 distance to the one on the left)

k-Nearest Neighbors: Summary

- In **image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**
- The **K-Nearest Neighbors** classifier predicts labels based on nearest training examples
- Distance metric and K are **hyperparameters**
- Choose hyperparameters using the **validation set**; only run on the test set once at the very end!

Linear classifiers

Score function

class scores

Slides from Andrej Karpathy and Fei-Fei Li
http://vision.stanford.edu/teaching/cs231n/

Score function: f

Parametric approach

image parameters

$f(\mathbf{x}, \mathbf{W})$

10 numbers, indicating class scores

[32x32x3]
array of numbers 0...1
(3072 numbers total)

Parametric approach: Linear classifier

$$f(x, W) = Wx$$

3072x1

10x1

10x3072

10 numbers, indicating class scores

[32x32x3]
array of numbers 0...1

parameters, or “weights”

Parametric approach: Linear classifier

$f(x, W) = Wx$ (+b)

3072x1

10x1

10x1

10x3072

10 numbers, indicating class scores

[32x32x3]
array of numbers 0...1

parameters, or “weights”

Linear Classifier

define a **score function**

data (image)

$$f(x_i, W, b) = Wx_i + b$$

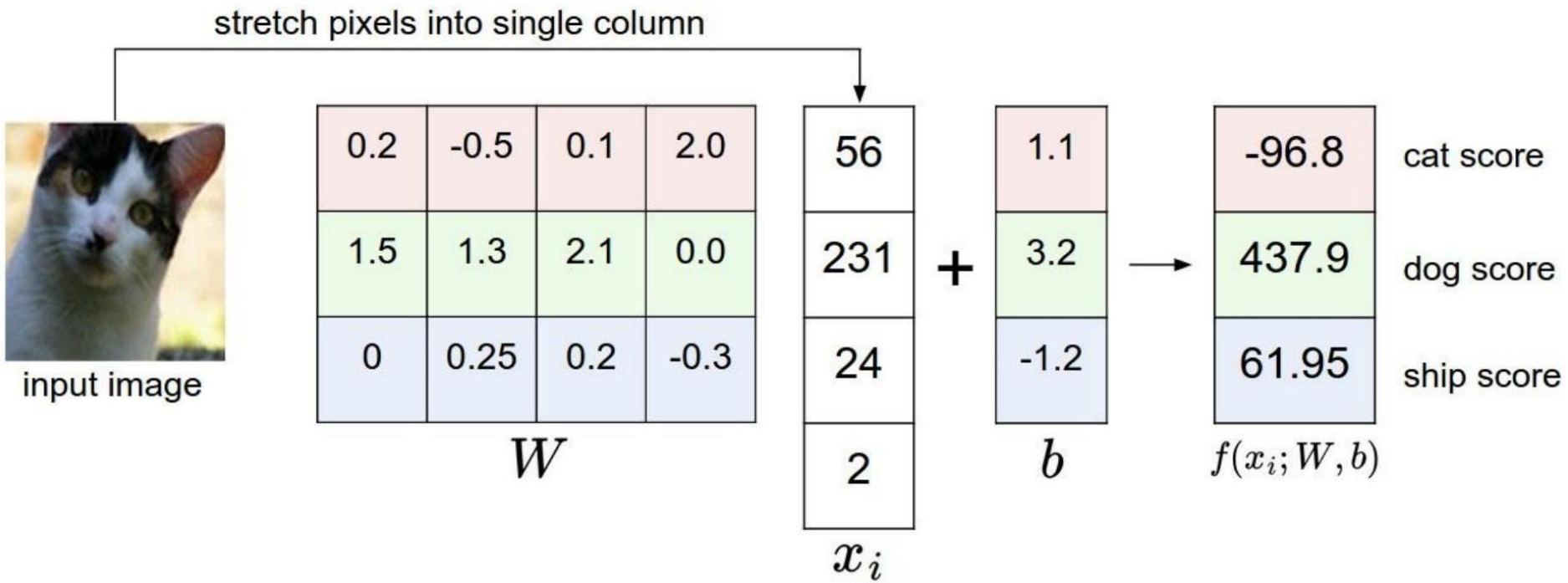
class scores

"weights"

"bias vector"

"parameters"

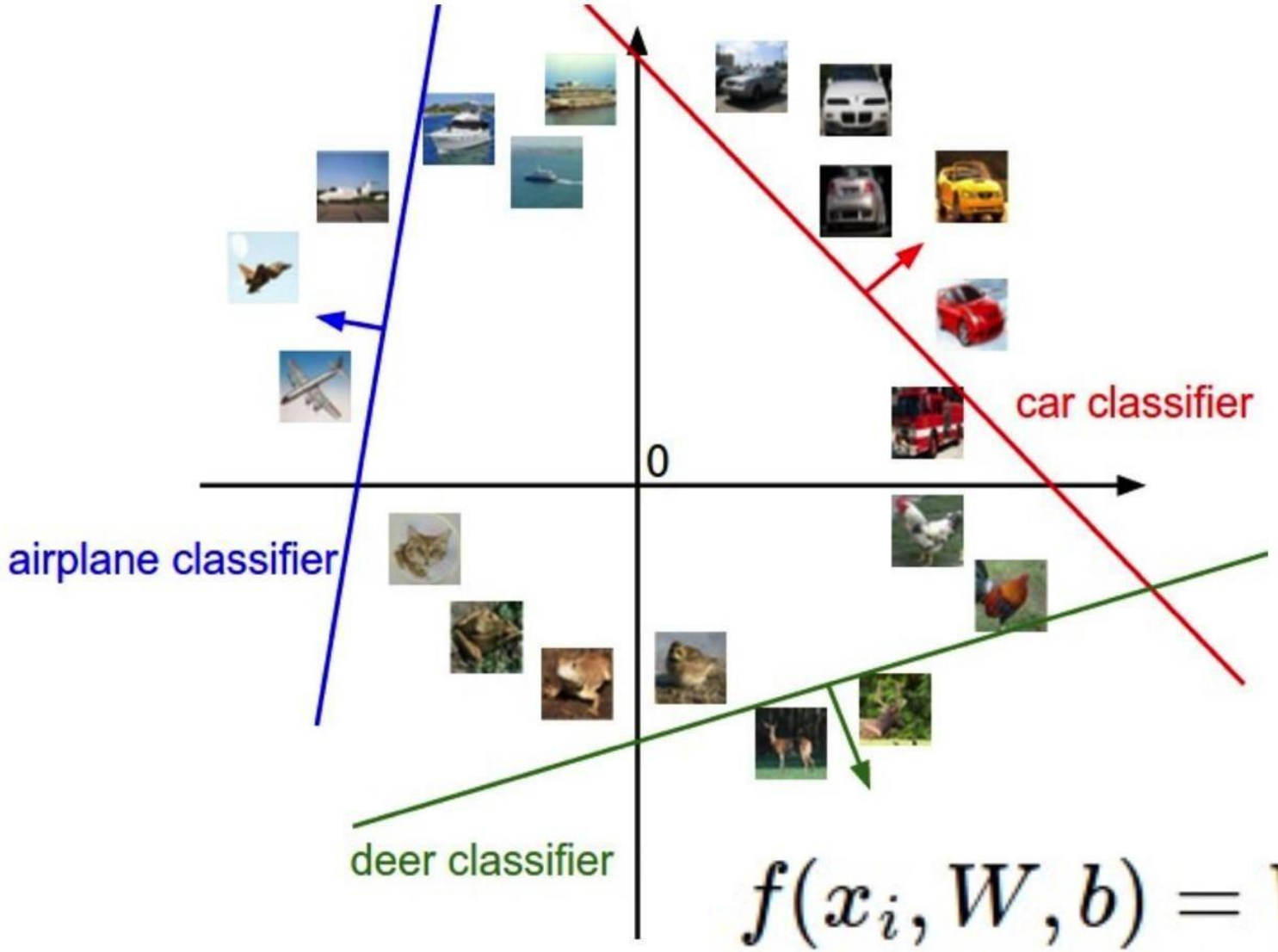
Example with an image with 4 pixels, and 3 classes (cat/dog/ship)



Interpretation: Template matching

$$f(x_i, W, b) = Wx_i + b$$

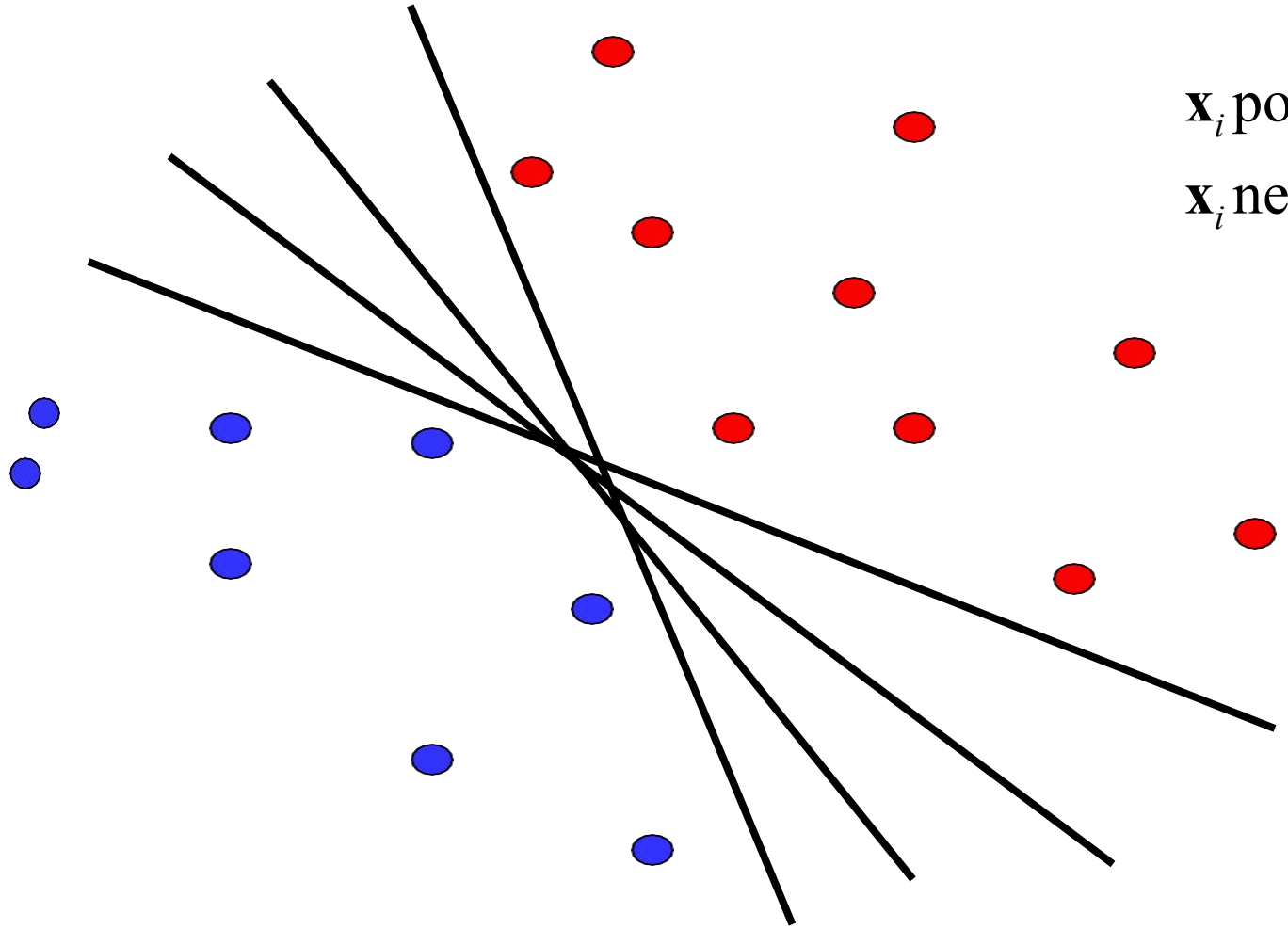
Geometric Interpretation



$$f(x_i, W, b) = Wx_i + b$$

Linear classifiers

- Find linear function (*hyperplane*) to separate positive and negative examples



$$\mathbf{x}_i \text{ positive :} \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 0$$

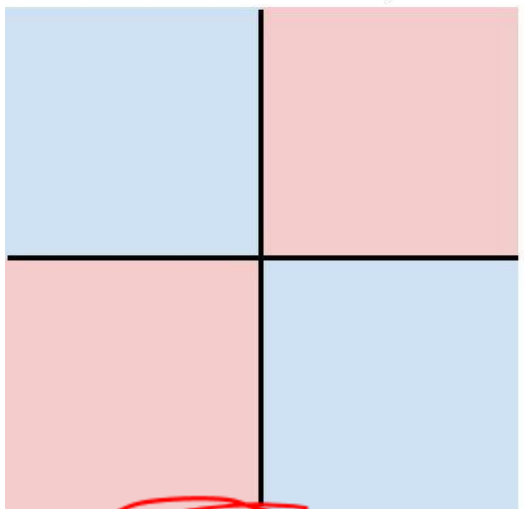
$$\mathbf{x}_i \text{ negative :} \quad \mathbf{x}_i \cdot \mathbf{w} + b < 0$$

Which hyperplane is best? We will come back to this later

Hard cases for a linear classifier

Class 1:
First and third quadrants

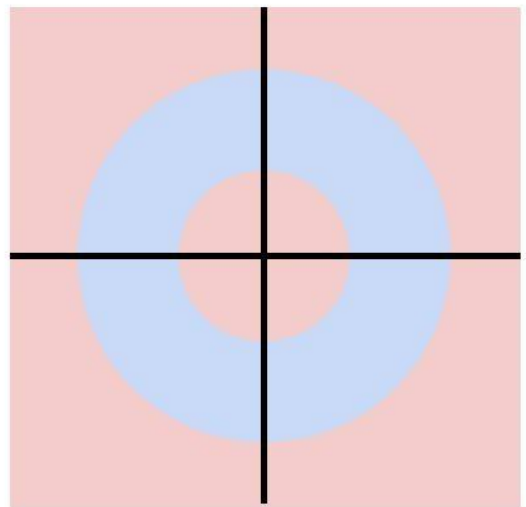
Class 2:
Second and fourth quadrants



XOR

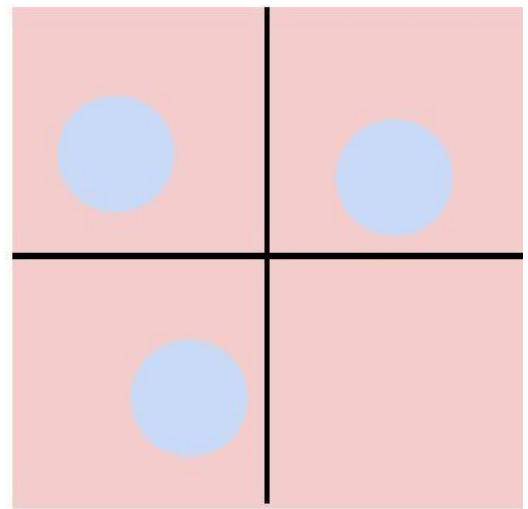
Class 1:
1 <= L2 norm <= 2

Class 2:
Everything else



Class 1:
Three modes

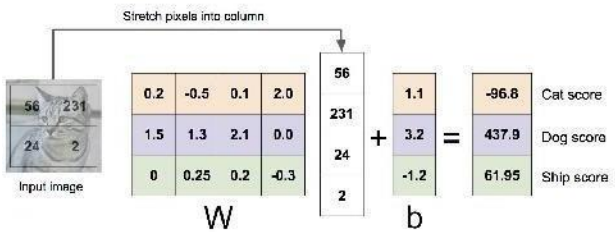
Class 2:
Everything else



Linear Classifier: Three Viewpoints

Algebraic Viewpoint

$$f(x,W) = Wx$$

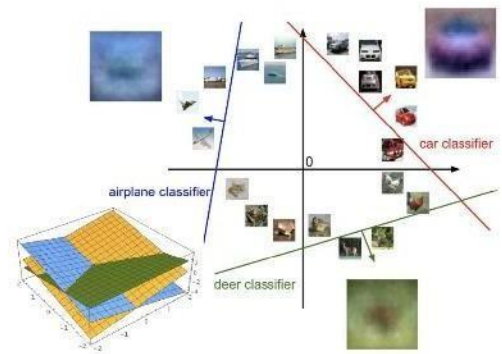


Visual Viewpoint

One template per class

Geometric Viewpoint

Hyperplanes cutting up space



So far: Defined a (linear) score function $f(x,W) = Wx + b$

Example class scores for 3 images for some W:

How can we tell whether this W is good or bad?



airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Recap

- Learning methods
  - k-Nearest Neighbors
  - **Linear classification**

- Classifier outputs a **score function** giving a score to each class
- how do we define how good a classifier is based on the training data? (Spoiler: define a *loss function*)

Linear classification



airplane	-3.45	-0.51	3.42
automobile	-8.87	**6.04**	4.64
bird	0.09	5.31	2.65
cat	**2.9**	-4.22	5.1
deer	4.48	-4.19	2.64
dog	8.02	3.58	5.55
frog	3.78	4.49	**-4.34**
horse	1.06	-4.37	-1.5
ship	-0.36	-2.09	-4.79
truck	-0.72	-2.93	6.14

Output scores

TODO:

1. Define a **loss function** that quantifies our unhappiness with the scores across the training data.

2. Come up with a way of efficiently finding the parameters that minimize the loss function. **(optimization)**

Suppose: 3 training examples, 3 classes.
With some W the scores $f(x, W) = Wx$ are:



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

A **loss function** tells how good our current classifier is

Given a dataset of examples

$$\{(x_i, y_i)\}_{i=1}^{N}$$

Where x_i is image and
y_i is (integer) label

Loss over the dataset is a sum of loss over examples:

$$L = \frac{1}{N} \sum_i L_i(f(x_i, W), y_i)$$

Loss function, cost/objective function

- Given ground truth labels (y_i), scores $f(x_i, \mathbf{W})$
  - how unhappy are we with the scores?

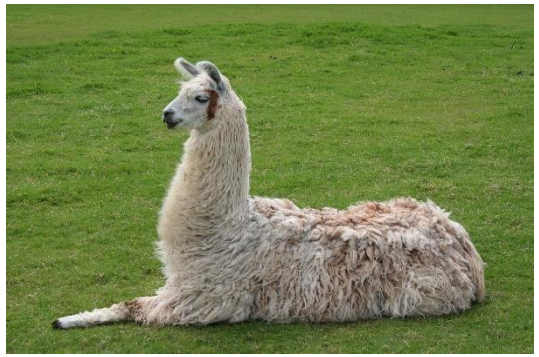
- Loss function or objective/cost function measures unhappiness

- During training, **want to find the parameters W that minimizes the loss function**

Simpler example: binary classification

- Two classes (e.g., “cat” and “not cat”)
  - AKA “positive” and “negative” classes

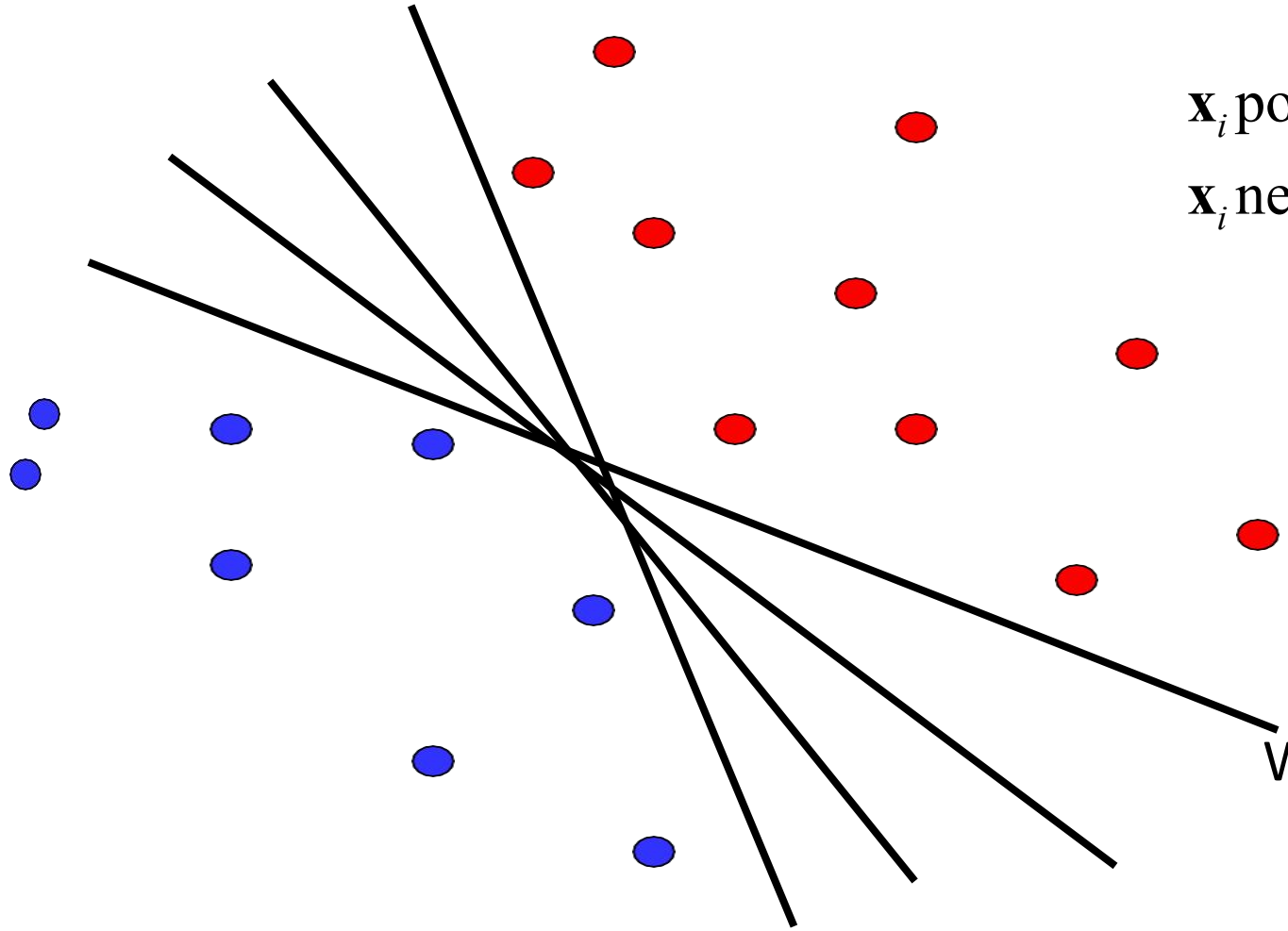
cat



not cat

Linear classifiers

- Find linear function (*hyperplane*) to separate positive and negative examples



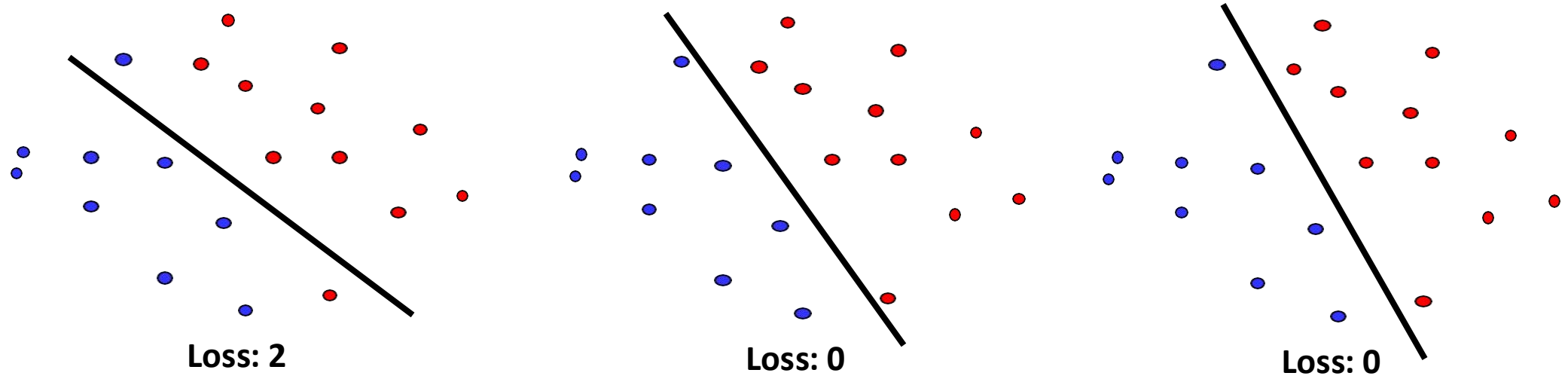
$$\mathbf{x}_i \text{ positive :} \quad \mathbf{x}_i \cdot \mathbf{w} + b \geq 0$$

$$\mathbf{x}_i \text{ negative :} \quad \mathbf{x}_i \cdot \mathbf{w} + b < 0$$

Which hyperplane is best? We need a **loss function** to decide

What is a good loss function?

- One possibility
  - Number of misclassified examples



  - Problems: discrete, can't break ties
  - We want the loss to lead to *good generalization*
  - We want the loss to work for more than 2 classes

Softmax classifier

$$f(x_i, W) = Wx_i$$ (score function)

$$\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}$$ softmax function

Example with three classes:

$$[1, -2, 0] \rightarrow [e^1, e^{-2}, e^0] = [2.71, 0.14, 1] \rightarrow [0.7, 0.04, 0.26]$$

Interpretation: squashes values into *probabilities* ranging from 0 to 1

$$P(y_i \mid x_i; W)$$

Cross-entropy loss

$$f(x_i, W) = Wx_i$$ (score function)

Losses

- Cross-entropy loss is just one possible loss function
  - One nice property is that it reinterprets scores as probabilities, which have a natural meaning
- SVM (max-margin) loss functions also used to be popular
  - But currently, cross-entropy is the most common classification loss

Summary

- Have score function and loss function
  - Currently, score function is based on linear classifier
  - Next, will generalize to convolutional neural networks
- Find W and b to minimize loss

$$L = \frac{1}{N}\sum_i -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right) + \lambda \sum_k \sum_l W_{k,l}^2$$

- ***Additional Readings and References:***

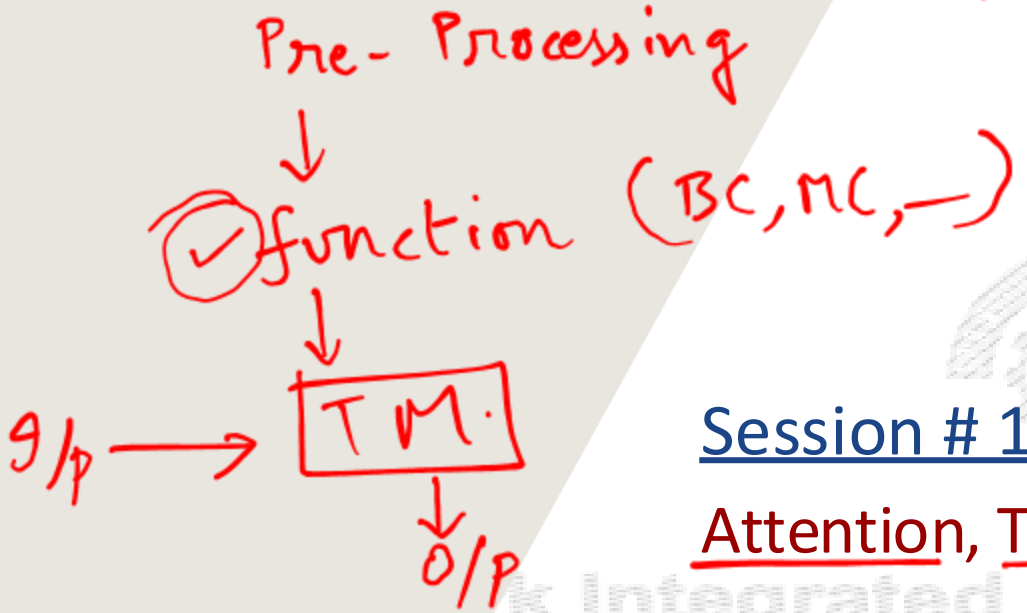
- Stanford CS231N
  – http://cs231n.stanford.edu/
- https://www.cs.toronto.edu/~kriz/cifar.html

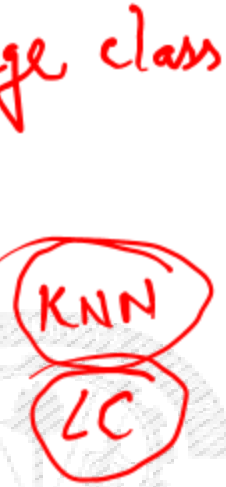
Hands on
https://github.com/mithunkumarsr/LearnComputerVisionWithMithun/blob/main/CV9_Image_Classification.ipynb

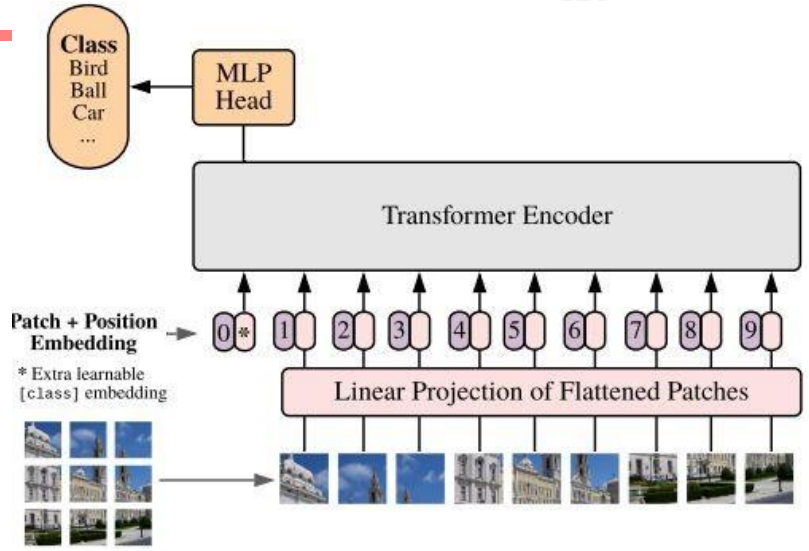
Thank you

Computer Vision

S1-24_AIMLCZG525, M.Tech (AIML)



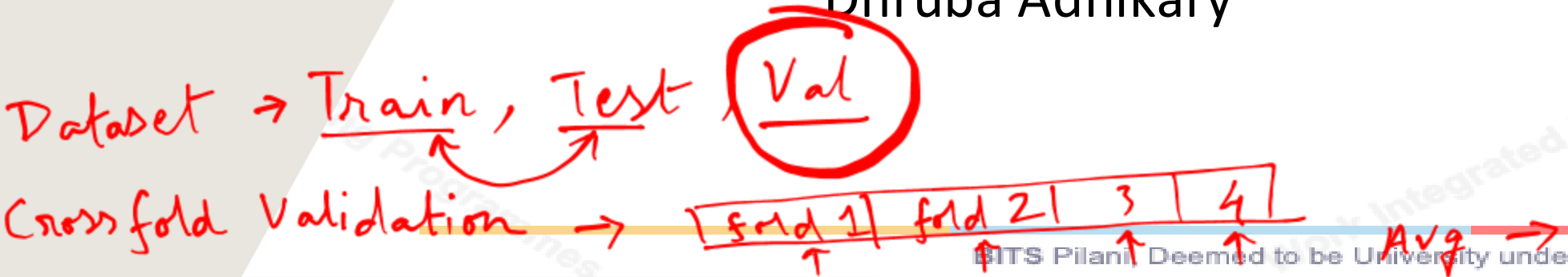




Session # 10:

Attention, Transformers & Vision Transformers (ViT)

Dhruba Adhikary

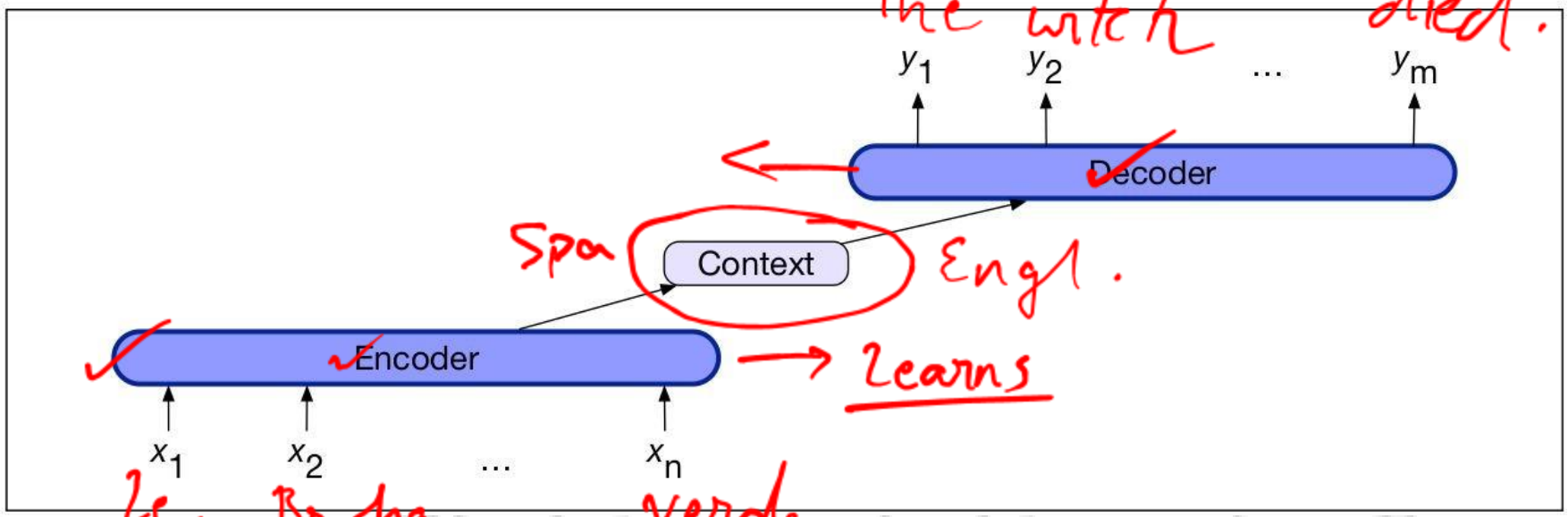


Topics

- Attention (Review)
- Transformers (Review)
- Vision Transformer (ViT)
- Applications of ViT in Computer Vision
- Popular ViT Variants

Acknowledgement: Slide Materials adopted from - Intro to Computer Vision (Cornell Tech); Noah Snavely

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation

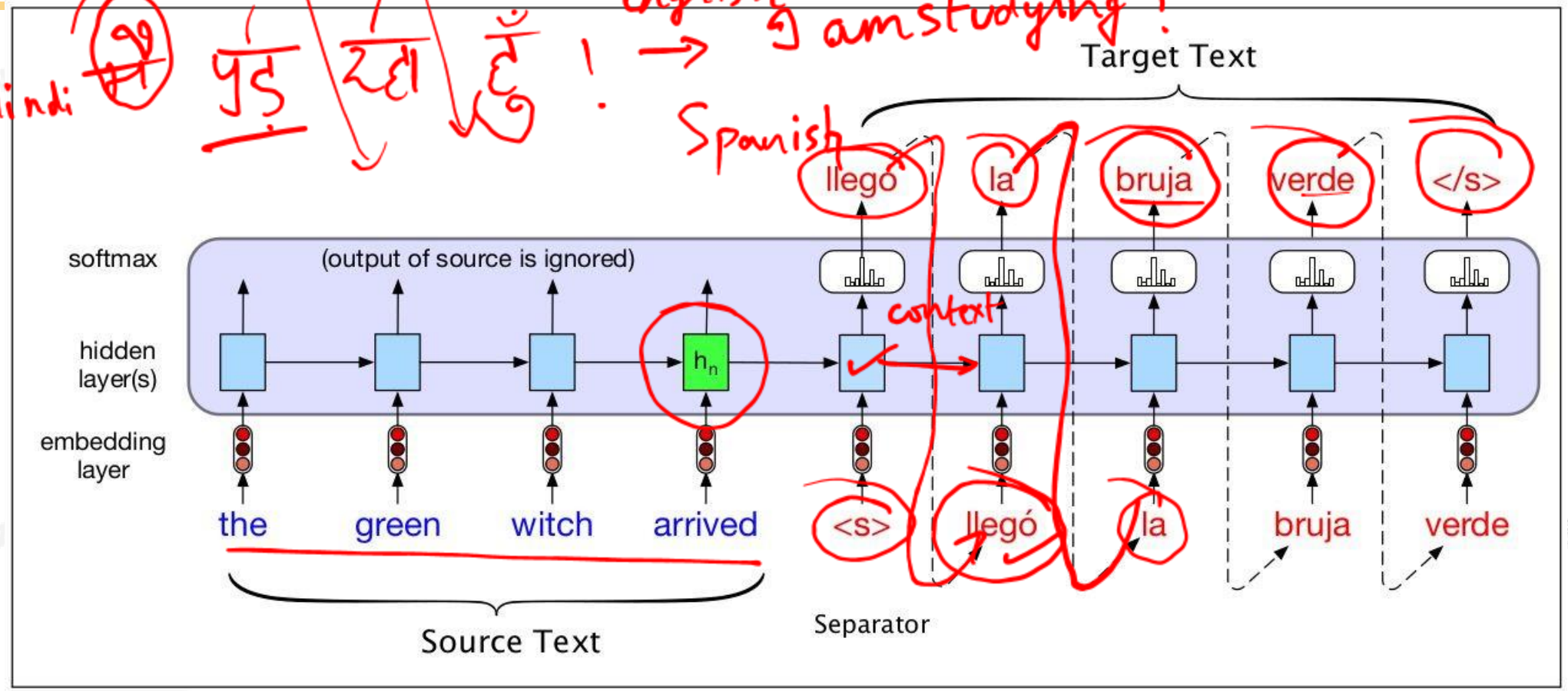


Components:

1. Encoder
2. Context Vector, c
3. Decoder

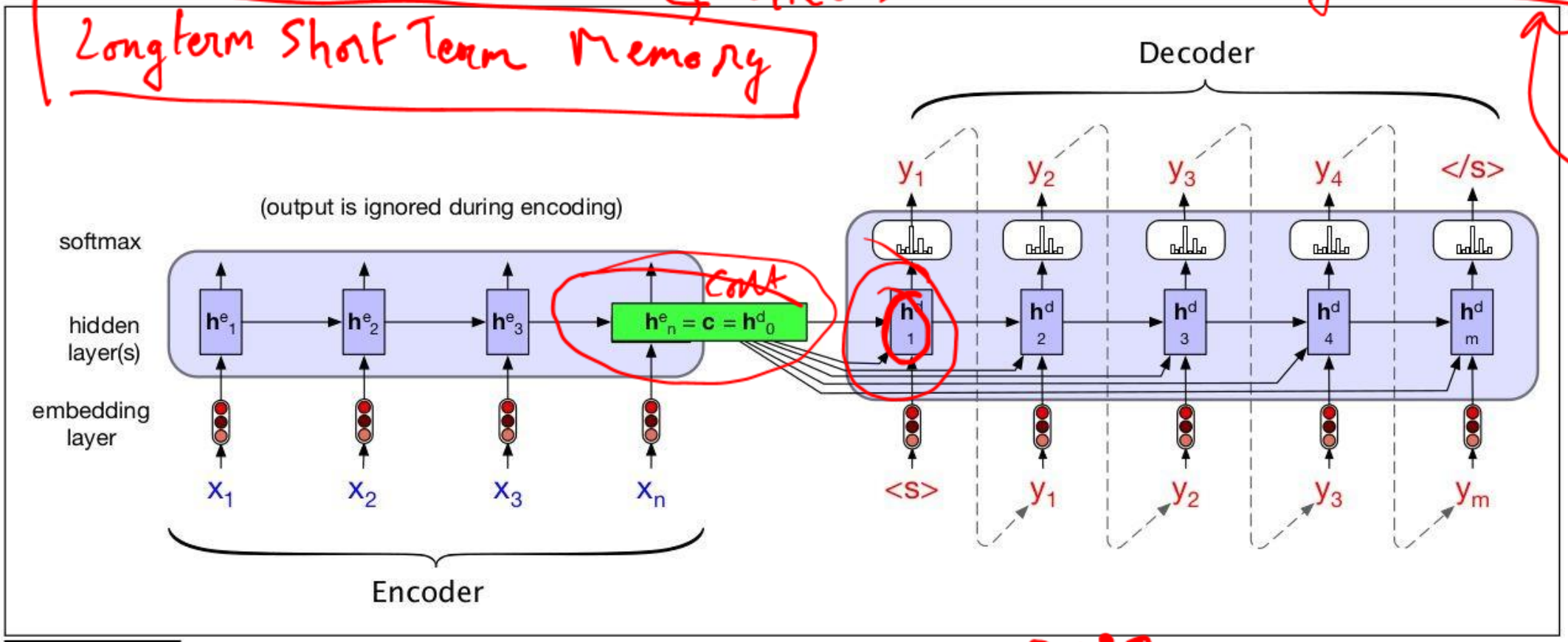
Note: The context is a function of the hidden representations of the input, and may be used by the decoder in a variety of ways

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



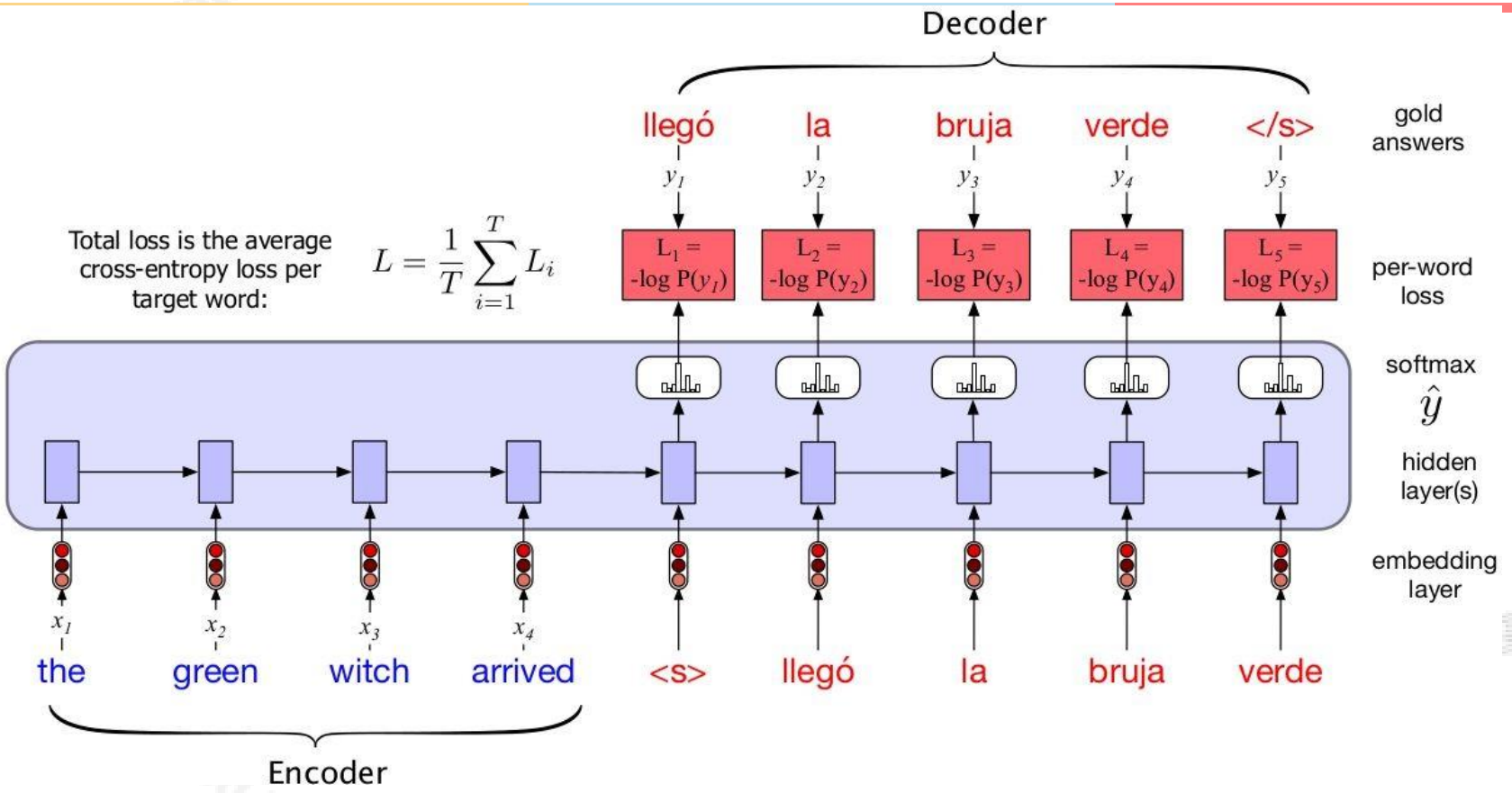
We make the context vector c available to more than just the first decoder hidden state, to ensure that the influence of the context vector, c, doesn't wane as the output sequence is generated.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



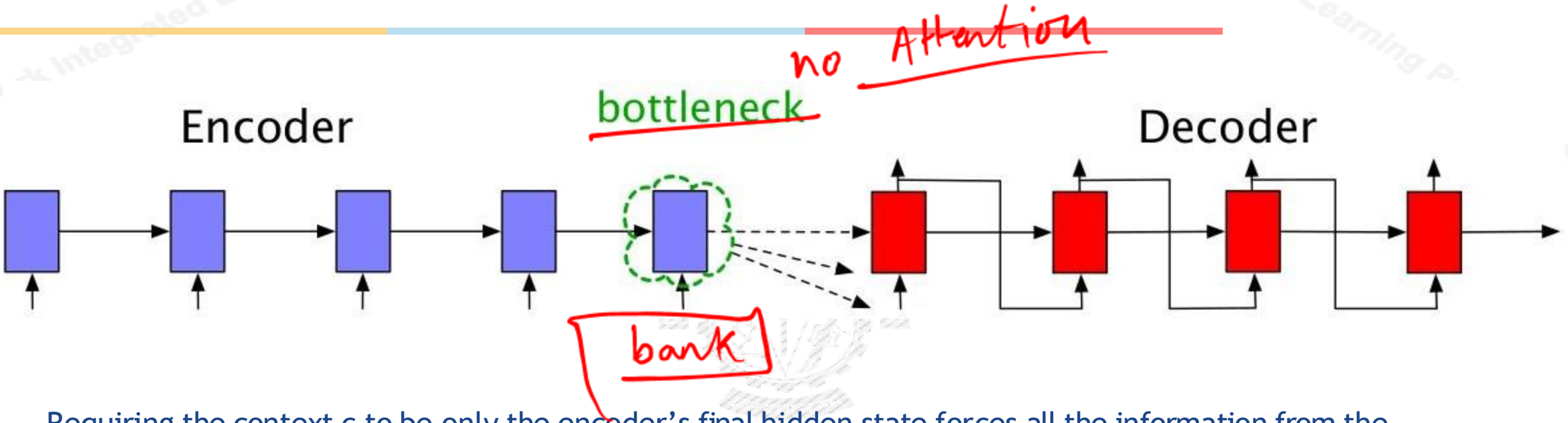
Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



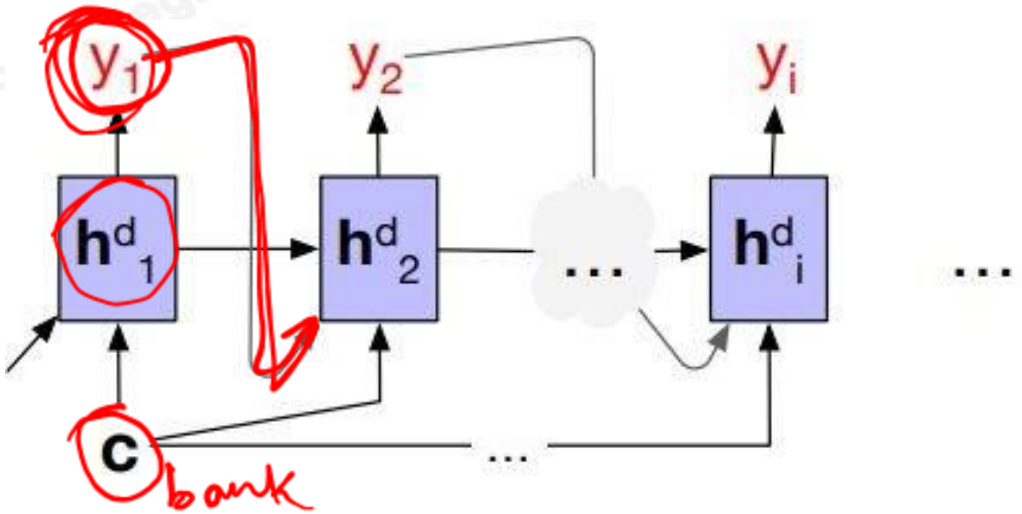
In the decoder we usually don't propagate the model's softmax outputs ŷt , but use teacher forcing. We compute the softmax output distribution over ŷ in the decoder, which can then be averaged to compute a loss for the sentence.

(Review) Encoder - Decoder Arch for Seq. to Seq. Translation



Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational bottleneck.

(Review) Attention !



Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

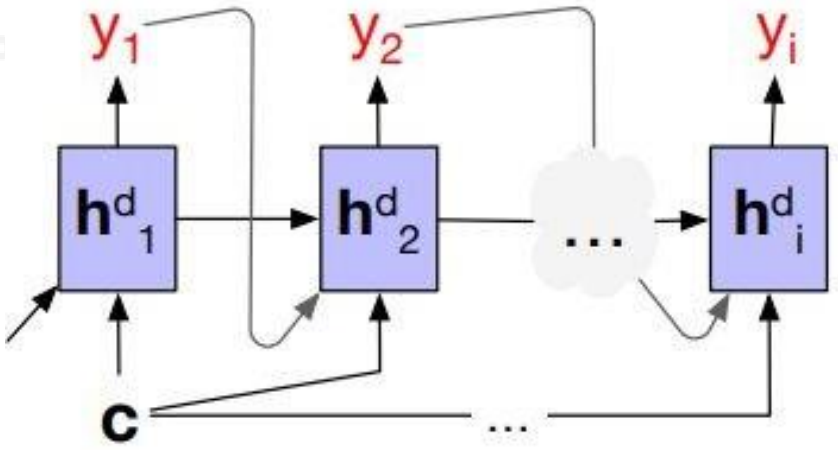
$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

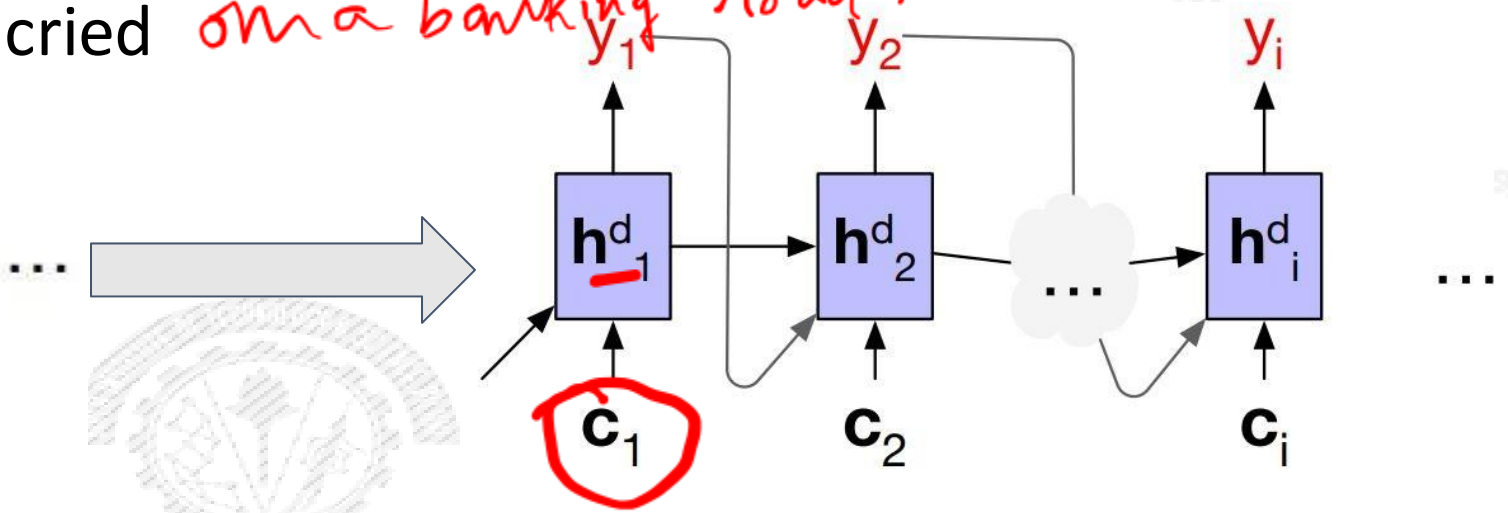
bank → financial I → 0.9
river → 0.85
curvature of road → 0.05

bank of river → 0.1, 0.85, 0.05

(Review) Attention !

He went to the bank to find his bank account was empty and then he went to the river bank and cried





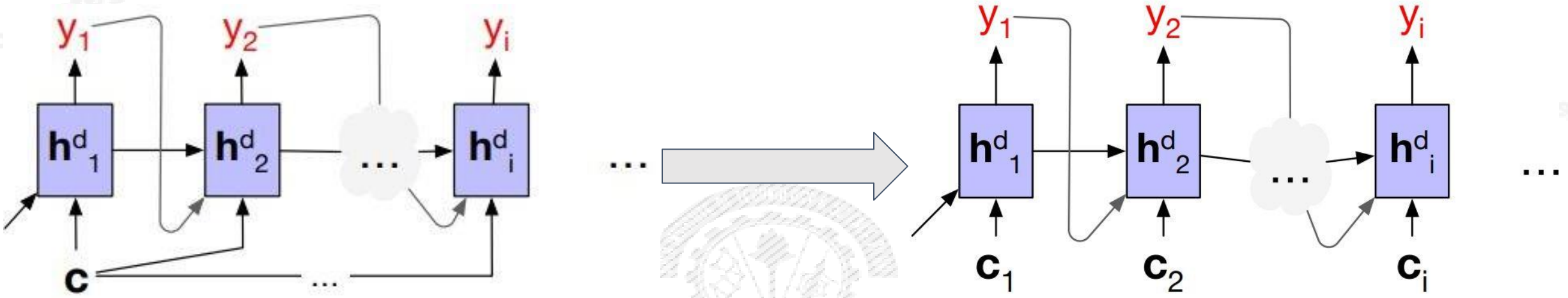
Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

(Review) Attention !



Without attention, a decoder sees the same context vector , which is a static function of all the encoder hidden states

With attention, decoder to sees a different, dynamic, context, which is a function of all the encoder hidden states

$$\mathbf{h}_t^d = g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c})$$

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$

With attention, decoder gets information from all the hidden states of the encoder, not just the last hidden state of the encoder Each context vector is obtained by taking a weighted sum of all the encoder hidden states.

The weights focus on ('attend to') a particular part of the source text that is relevant for the token the decoder is currently producing

(Review) Attention !

Step -1 : Find out how relevant each encoder state is to the present decoder state $\mathbf{h}_{i-1}^{d}$

Compute a score of similarity between $\mathbf{h}_{i-1}^{d}$ and all the encoder states : $score(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e})$

Dot Product Attention : $\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}) = \mathbf{h}_{i-1}^{d} \cdot \mathbf{h}_{j}^{e}$

Step -2 : Normalize all the scores with softmax to create a vector of weights, $\alpha_{i,j}$

$\alpha_{i,j}$ indicates the proportional relevance of each encoder hidden state j to the prior hidden decoder state, $\mathbf{h}_{i-1}^{d}$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}) \ \forall j \in e)$$

$$= \frac{\exp(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{j}^{e}))}{\sum_{k} \exp(\text{score}(\mathbf{h}_{i-1}^{d}, \mathbf{h}_{k}^{e}))}$$

(Review) Attention !

Step -3 : Given the distribution in α, compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

(Review) Attention !

Step -3 : Given the distribution in α, compute a fixed-length context vector for the current decoder state by taking a weighted average over all the encoder hidden states

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

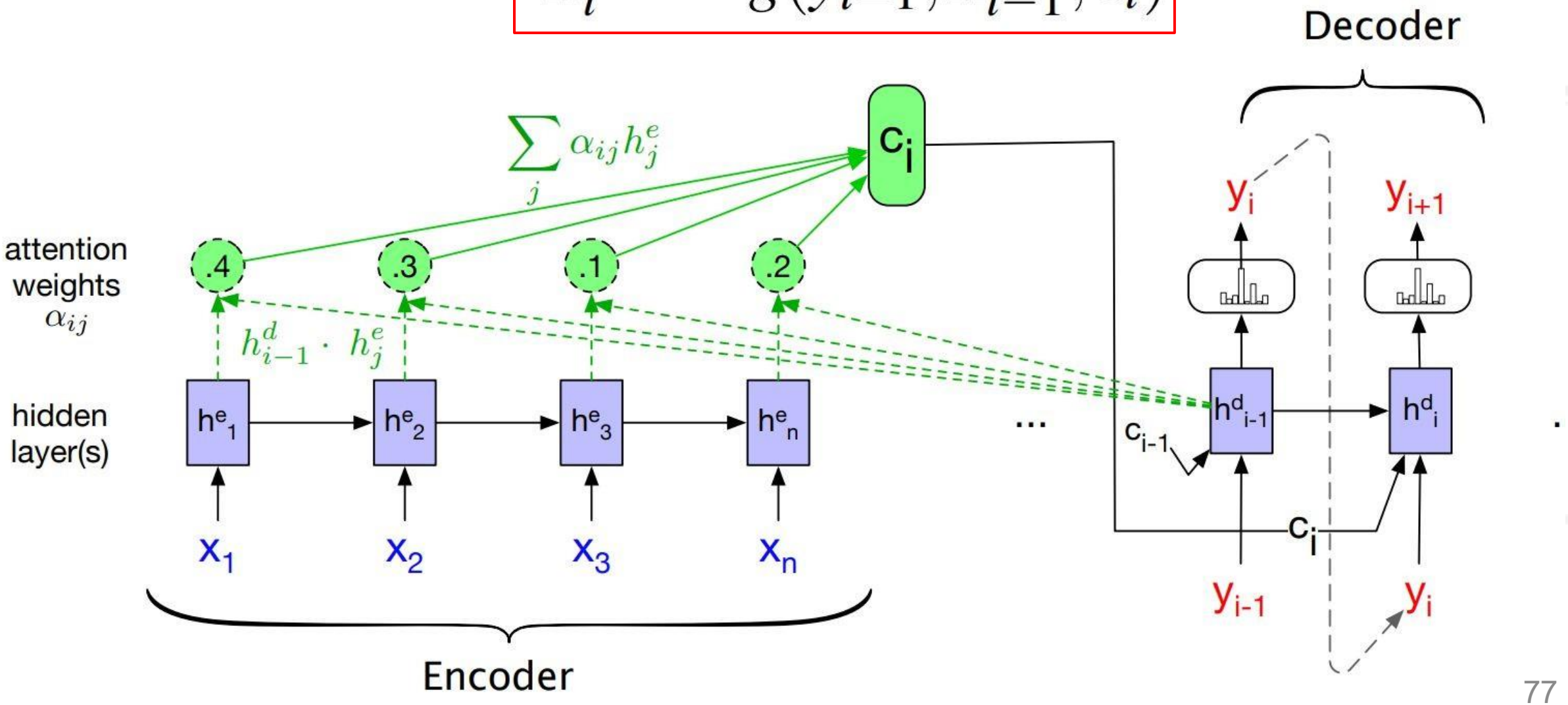
Plus : In step-1, we can get a more powerful scoring function by parameterizing the score with its own set of weights, W_s:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{t-1}^d \mathbf{W}_s \mathbf{h}_j^e$$

W_s , is trained during normal end-to-end training,
W_s , gives the network the ability to learn which aspects of similarity between the decoder and encoder states are important to the current application.

(Review) Attention !

$$\mathbf{h}_i^d = g(\hat{y}_{i-1}, \mathbf{h}_{i-1}^d, \mathbf{c}_i)$$



(Review) Transformers

- 2017, NIPS, Vaswani et. al., Attention Is All You Need !!!
- Transformers map sequences of input vectors $(x_1,...,x_n)$ to sequences of output vectors $(y_1,...,y_n)$ of the same length
- Made up of transformer blocks in which the key component is self-attention layers

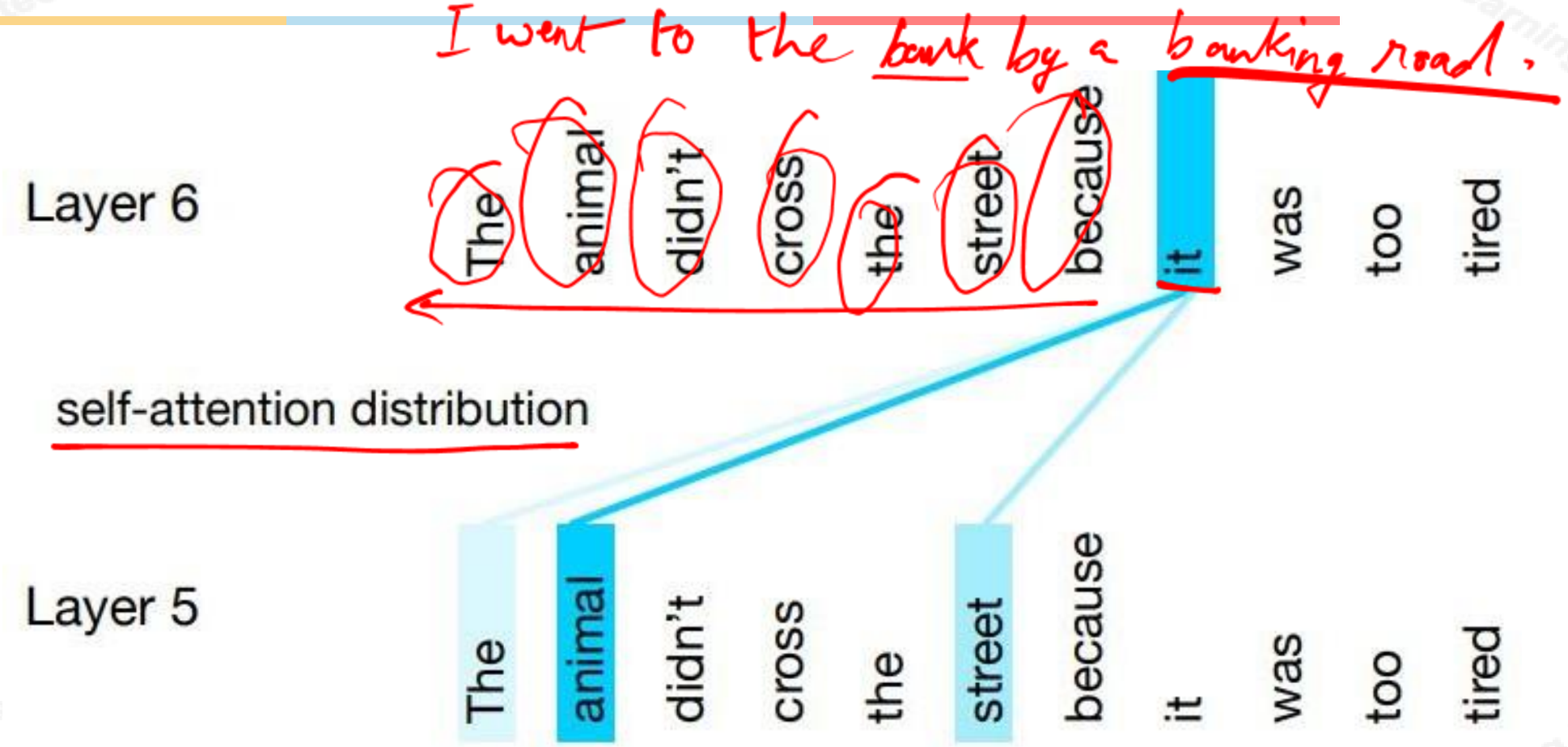
  [Self-attention allows a network to directly extract and use information from arbitrarily large contexts directly !!!]

- Transformers are not based on recurrent connections ⇒ Parallel implementations possible ⇒ Efficient to scale (comparing LSTM)

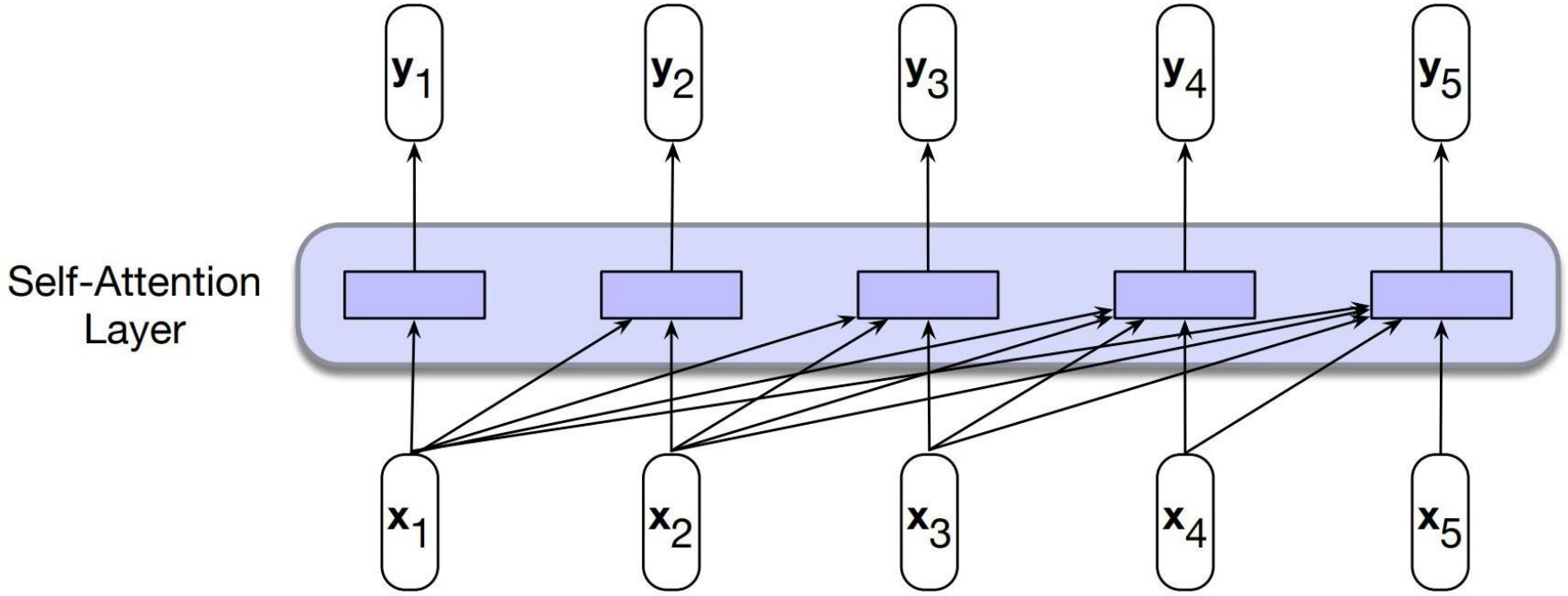
(Review) Self-Attention |Transformers

- Attention⇒ Ability to compare an item of interest to a collection of other items in a way that reveals their relevance in the current context.
- Self-attention ~
  - > Set of *comparisons* are to other elements *within a given sequence*
  - > Use these comparisons to compute an output for the current input

(Review) Self-Attention |Transformers



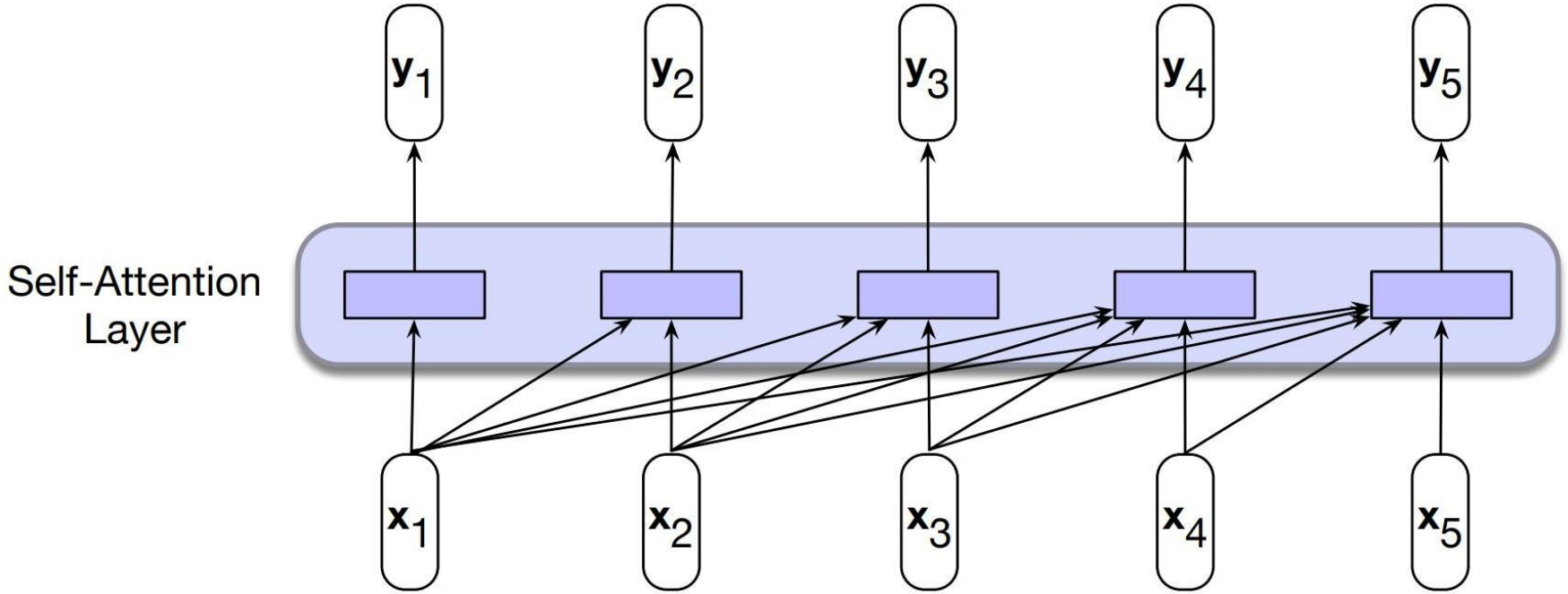
(Review) Self-Attention |Transformers



In processing each element of the sequence, the model attends to all the inputs up to, and including, the current one.

Unlike RNNs, the computations at each time step are independent of all the other steps and therefore can be performed in parallel.

(Review) Self-Attention |Transformers



$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i \cdot \mathbf{x}_j$$

$$\mathbf{y}_i = \sum_{j \le i} \alpha_{ij} \mathbf{x}_j$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \le i$$

$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^{i} \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \ \forall j \le i$$

(Review) Self-Attention |Transformers

- Let us understand how transformers uses self-attention !

(Review) Self-Attention |Transformers

$$\mathbf{W}^V, \mathbf{W}^K, \mathbf{W}^Q \in \mathbb{R}^{d \times d}$$

In Vaswani et al., 2017, d was 1024.

- Let us understand how transformers uses self-attention !

$$\mathbf{q}_i = \mathbf{W}^Q \mathbf{x}_i \qquad \mathbf{k}_i = \mathbf{W}^K \mathbf{x}_i \qquad \mathbf{v}_i = \mathbf{W}^V \mathbf{x}_i$$

Query, Q

As the current focus of attention when being compared to all of the other preceding inputs.

Key, K

In its role as a preceding input being compared to the current focus of attention.

Value, V

As a value used to compute the output for the current focus of attention

Three different roles each x_i (input embedding) , in the computation of self attention

(Review) Self-Attention |Transformers

$$\mathbf{W}^V, \mathbf{W}^K, \mathbf{W}^Q \in \mathbb{R}^{d \times d}$$

In Vaswani et al., 2017, d was 1024.

- Let us understand how transformers uses self-attention !

$$\mathbf{q}_i = \mathbf{W}^{\mathbf{Q}}\mathbf{x}_i \qquad \mathbf{k}_i = \mathbf{W}^{\mathbf{K}}\mathbf{x}_i \qquad \mathbf{v}_i = \mathbf{W}^{\mathbf{V}}\mathbf{x}_i$$

I am studying

$$\text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

The simple dot product can be an arbitrarily large; scaled dot-product is used in transformers;

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \le i$$

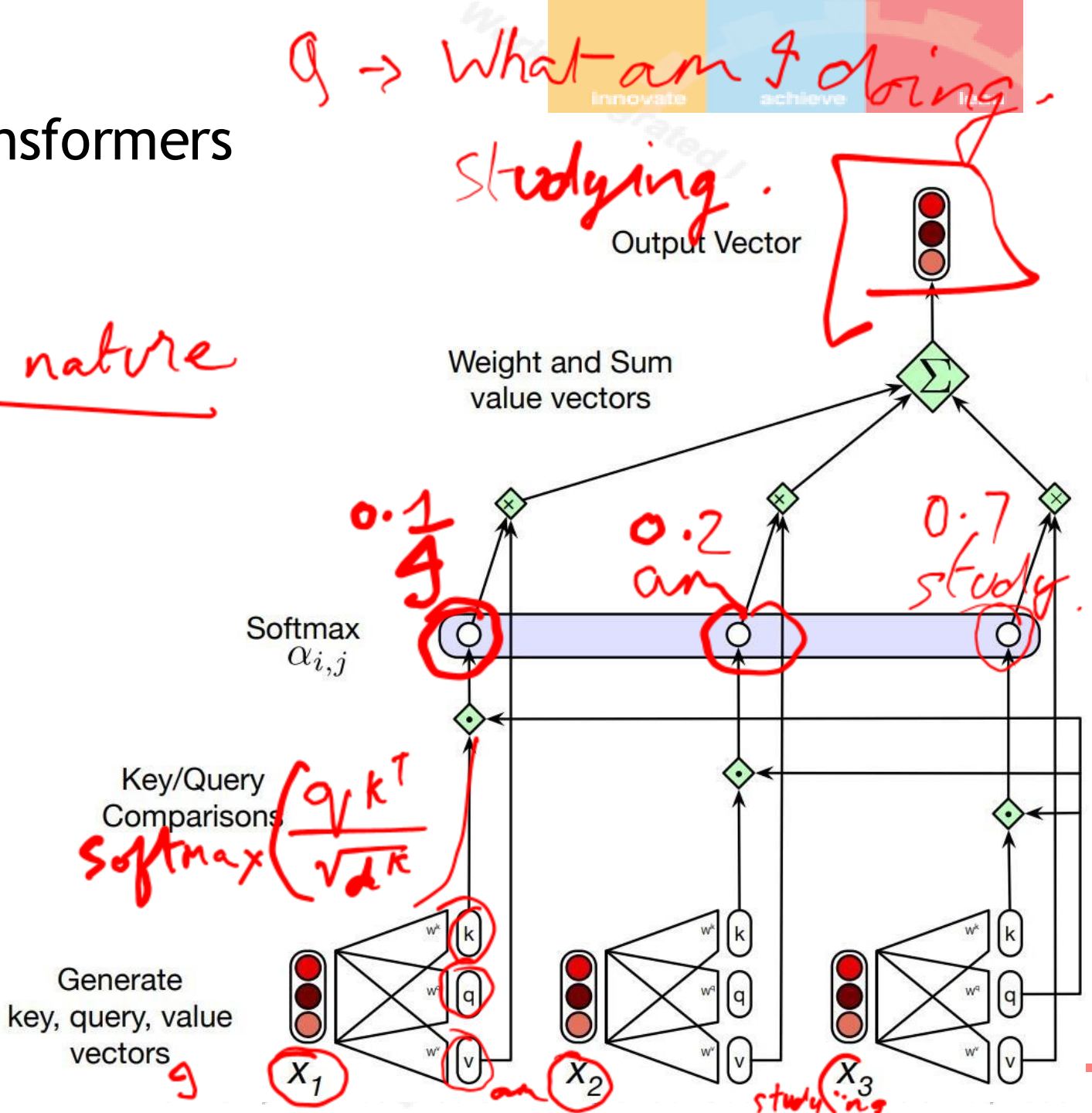
$$= \frac{\exp(\text{score}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{k=1}^{i} \exp(\text{score}(\mathbf{x}_i, \mathbf{x}_k))} \;\; \forall j \le i$$

$$\mathbf{y}_i = \sum_{j \le i} \alpha_{ij} \mathbf{v}_j$$

(Review) Self-Attention |Transformers

RNN's LSTMs sequtial in nature

- Each output, y_i , is computed independently
- Entire process can be parallelized



Calculating the value of y3, the third element of a sequence using causal (left-to-right) self-attention

(Review) Self-Attention |Transformers

- Pack the input embeddings of the N input tokens into a single matrix

$$\mathbf{X} \in \mathbb{R}^{N \times d}$$

> Each row of X is the embedding of one token of the input

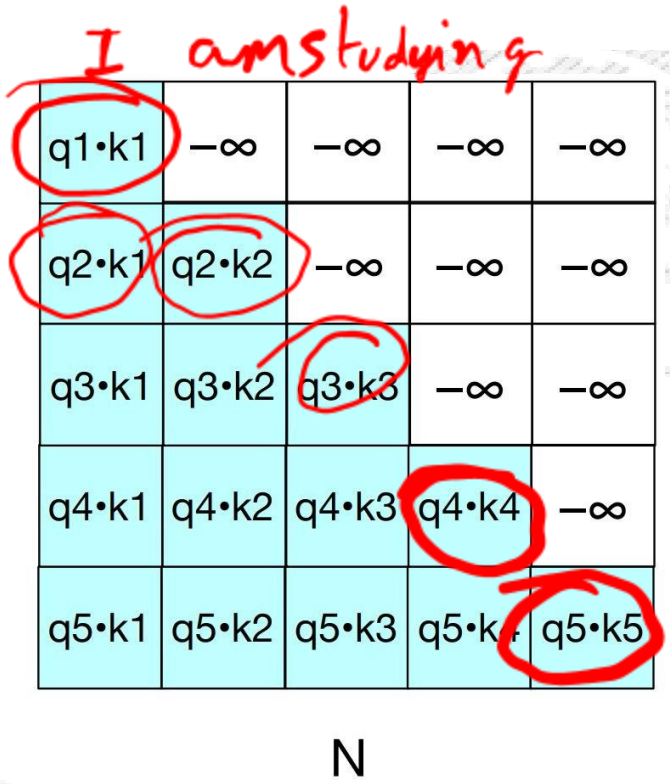
- Multiply X by the key, query, and value (dxd) matrices

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^{\mathbf{Q}}; \quad \mathbf{K} = \mathbf{X}\mathbf{W}^{\mathbf{K}}; \quad \mathbf{V} = \mathbf{X}\mathbf{W}^{\mathbf{V}}$$

(Review) Self-Attention |Transformers

$$\text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\intercal}{\sqrt{d_k}}\right)\mathbf{V}$$

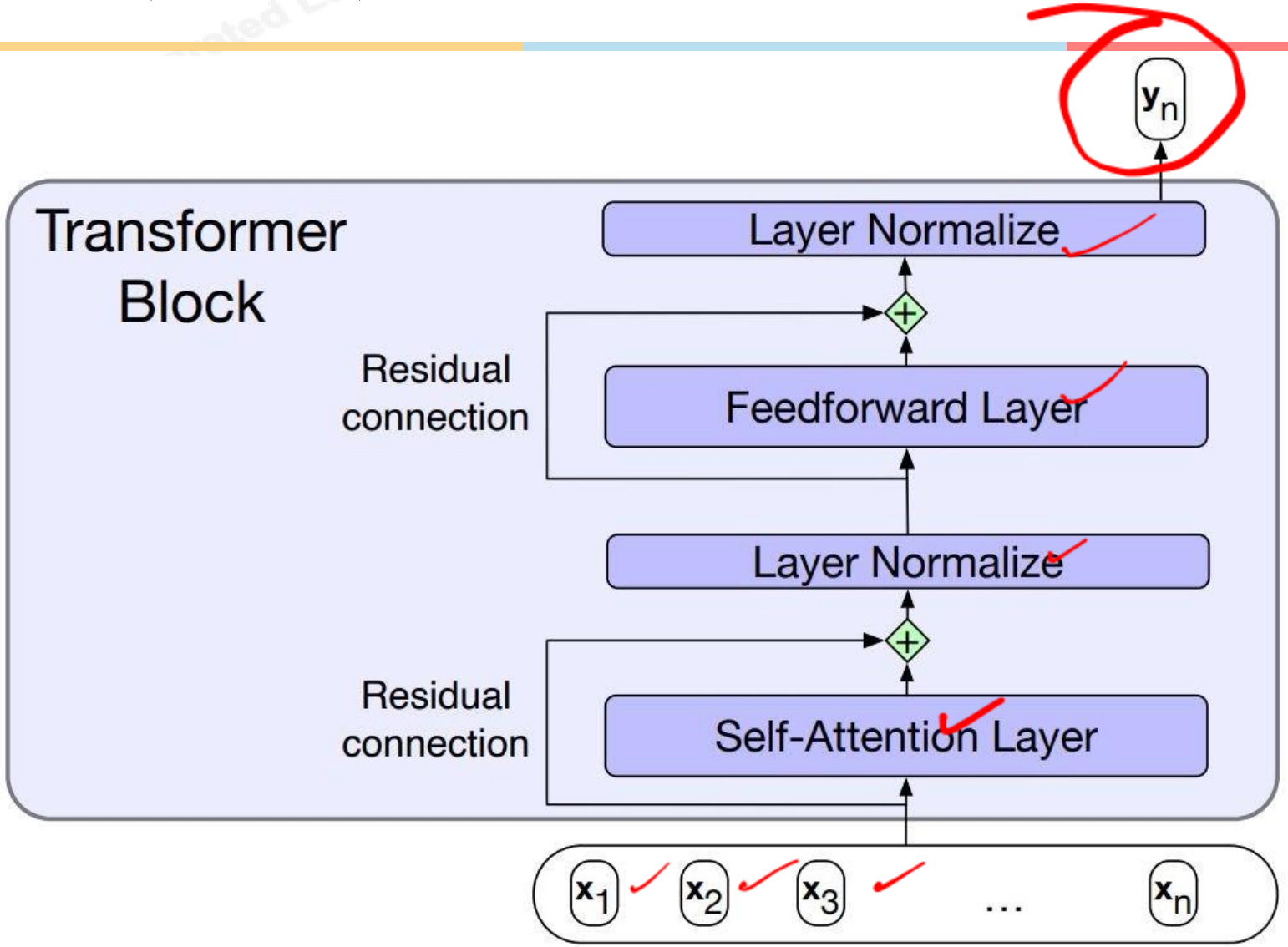
$\mathbf{QK}^\intercal$ Matrix



Note: Upper-triangle portion of the comparisons matrix zeroed out (set to −∞, which the softmax will turn to zero)

(Review) Transformer Blocks |Transformers

$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$$
$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$



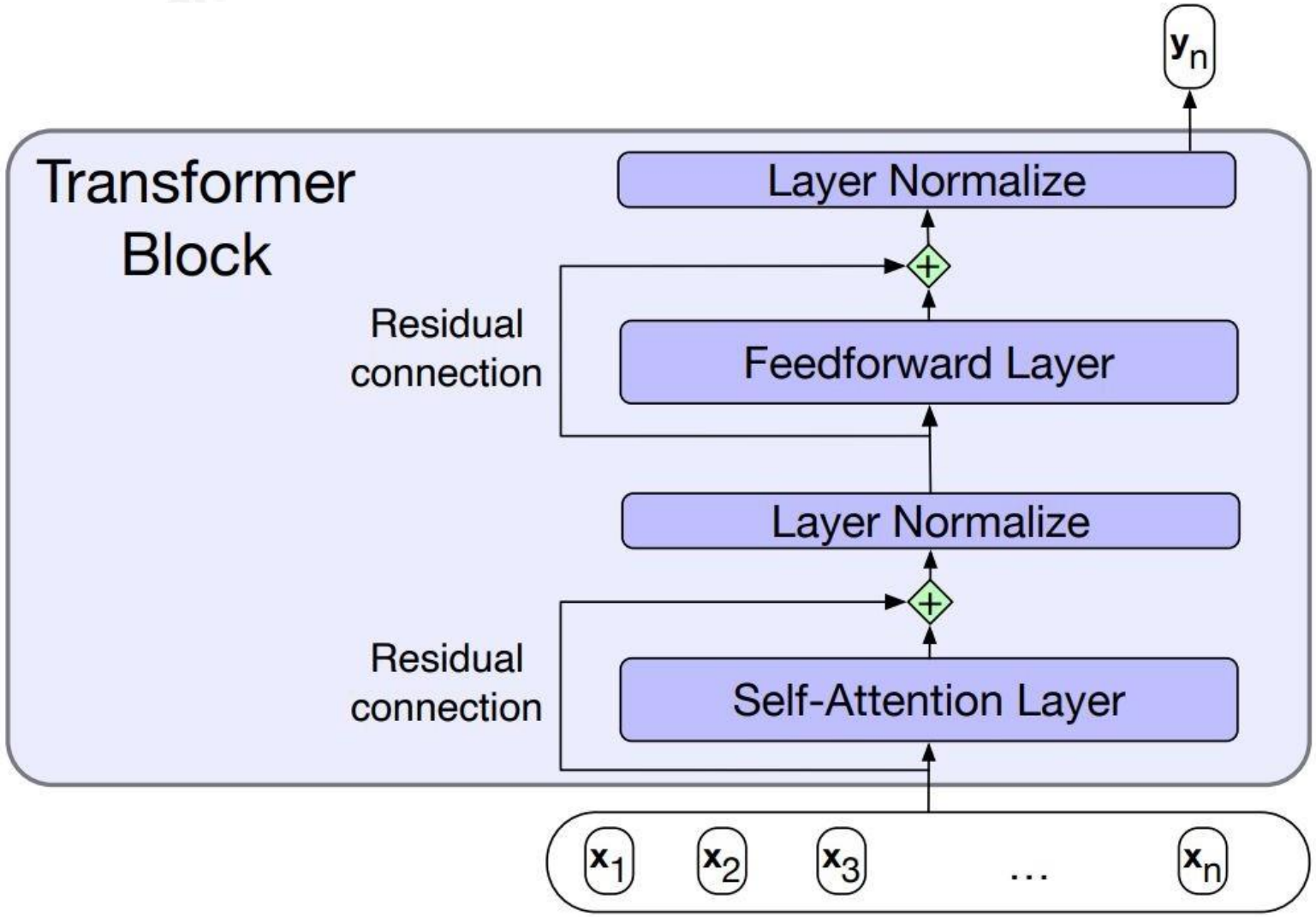
Summary

- Have score function and loss function
  - Currently, score function is based on linear classifier
  - Next, will generalize to convolutional neural networks
- Find W and b to minimize loss

$$L = \frac{1}{N}\sum_i -\log\left(\frac{e^{f_{y_i}}}{\sum_j e^{f_j}}\right) + \lambda \sum_k \sum_l W_{k,l}^2$$

$P(P/y/N)$

(Review) Transformer Blocks |Transformers



$$\mathbf{z} = \text{LayerNorm}(\mathbf{x} + \text{SelfAttention}(\mathbf{x}))$$
$$\mathbf{y} = \text{LayerNorm}(\mathbf{z} + \text{FFN}(\mathbf{z}))$$

LayerNorm:

$$\mu = \frac{1}{d_h}\sum_{i=1}^{d_h} x_i$$

$$\sigma = \sqrt{\frac{1}{d_h}\sum_{i=1}^{d_h}(x_i - \mu)^2}$$

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma}$$

$$\text{LayerNorm} = \gamma\,\hat{\mathbf{x}} + \beta$$

(Review) Multihead-Attention |Transformers

- Different words in a sentence can relate to each other in many different ways simultaneously

  >> A single transformer block to learn to capture all different kinds of parallel relations among its input is inadequate.

- Multihead self-attention layers

  >> Heads ⇒ sets of self-attention layers, that reside in parallel layers at the same depth in a model, each with its own set of parameters.

  >> Each head learn different aspects of the relationships that exist among inputs at the same level of abstraction

(Review) Multihead-Attention |Transformers

2017

- Different words in a sentence can relate to each other in many different ways simultaneously

1 million

- Multihead self-attention layers

>> A single transformer block to learn to capture all different kinds of parallel relations among its input is inadequate.

>> Heads ⇒ sets of self-attention layers, that reside in parallel layers at the same depth in a model, each with its own set of parameters.

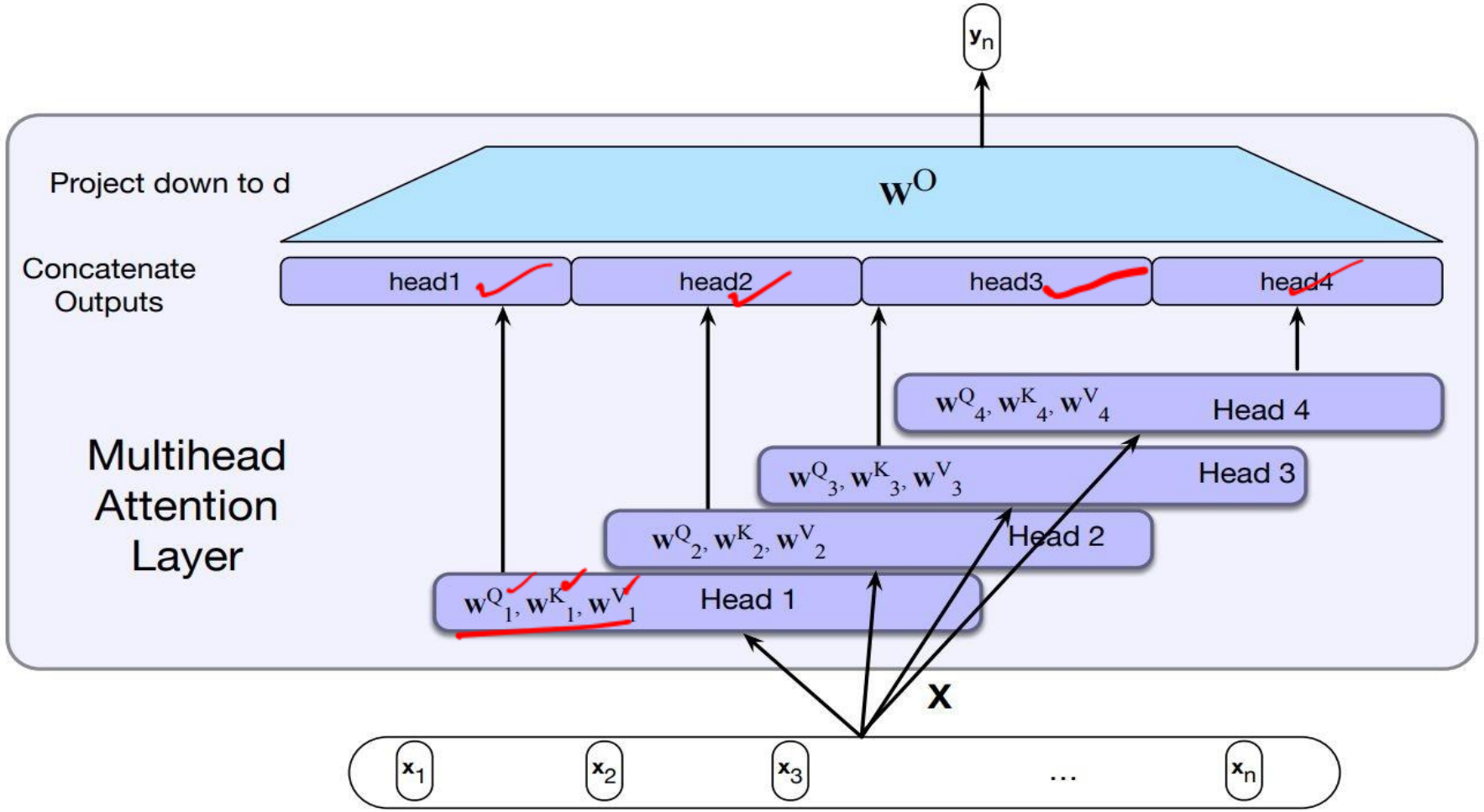
>> Each head learn different aspects of the relationships that exist among inputs at the same level of abstraction

$$\text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_i^Q ; \mathbf{K} = \mathbf{X}\mathbf{W}_i^K ; \mathbf{V} = \mathbf{X}\mathbf{W}_i^V$$

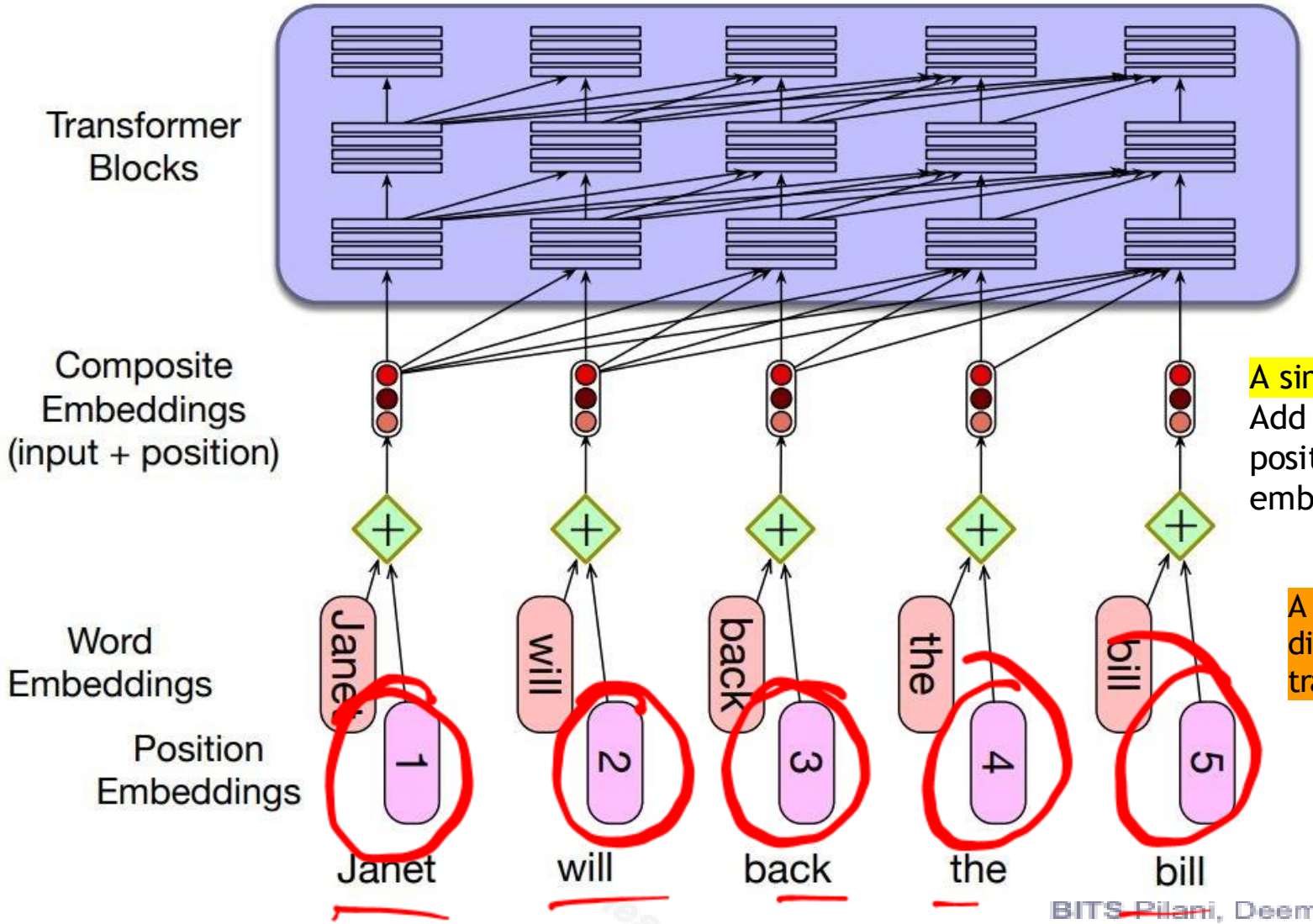
$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})$$

(Review) Multihead-Attention |Transformers



Each of the multihead self-attention layers is provided with its own set of key, query and value weight matrices.
The outputs from each of the layers are concatenated and then projected down to d, thus producing an output of the same size as the input so layers can be stacked.

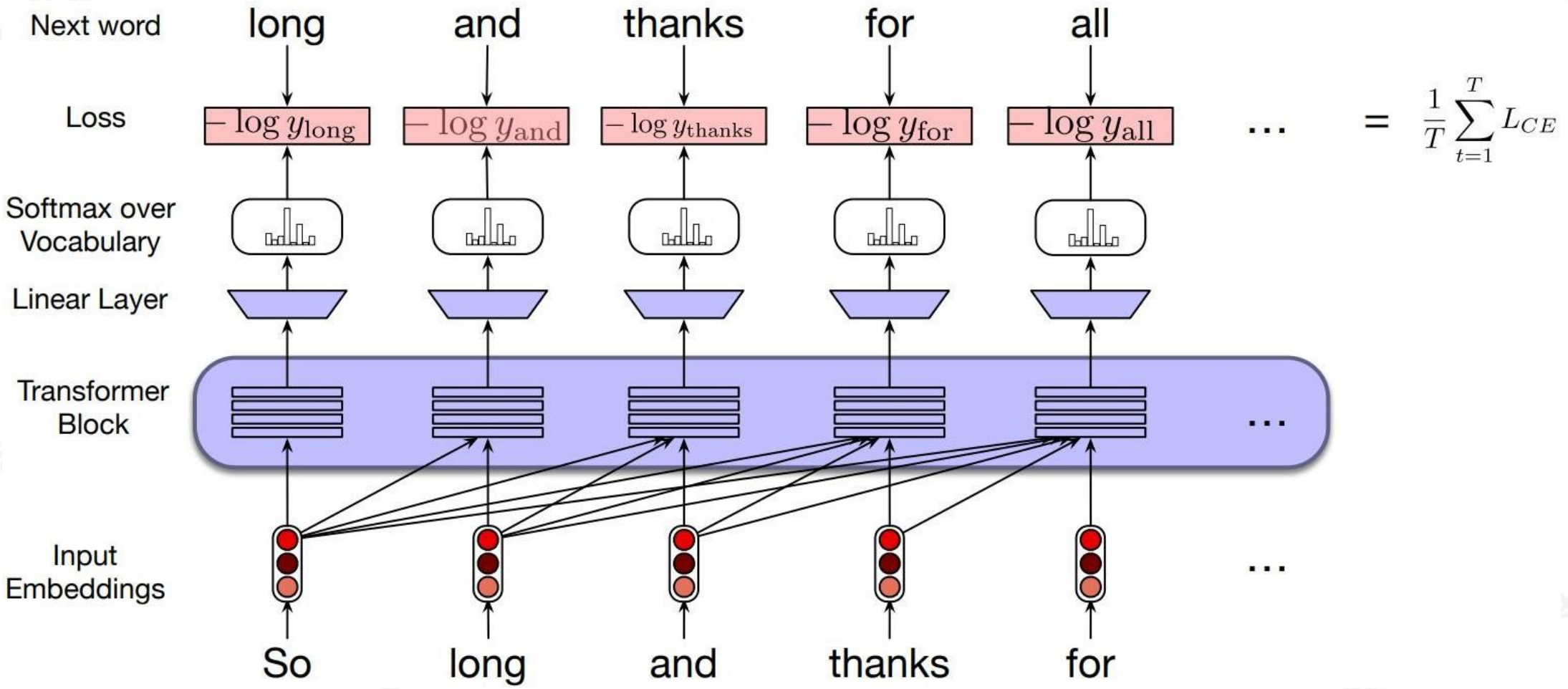
(Review) Positional Embeddings |Transformers



A simple way to model position:
Add an embedding representation of the absolute position to the input word embedding to produce a new embedding of the same dimensionality.

A combination of sine and cosine functions with differing frequencies was used in the original transformer work.

(Review) Transformers as Language Models



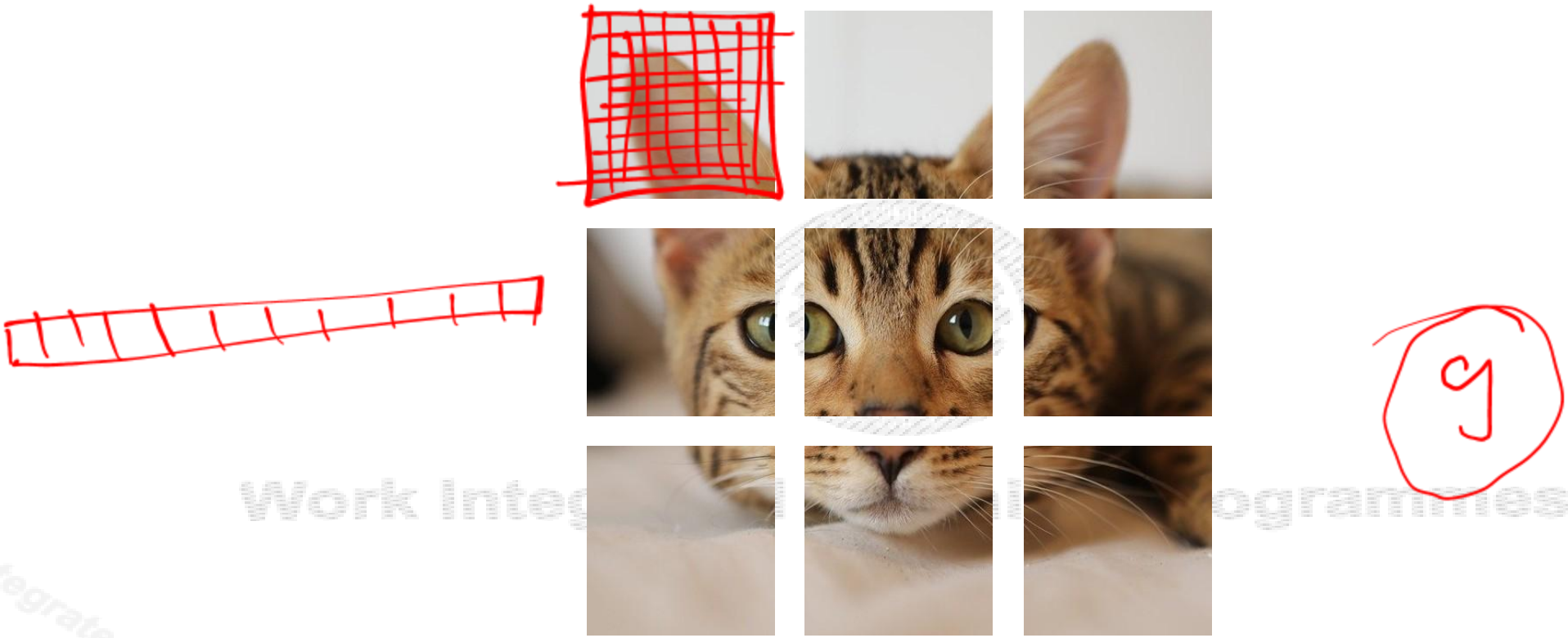
Vision Transformers

Vision Transformer (ViT)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

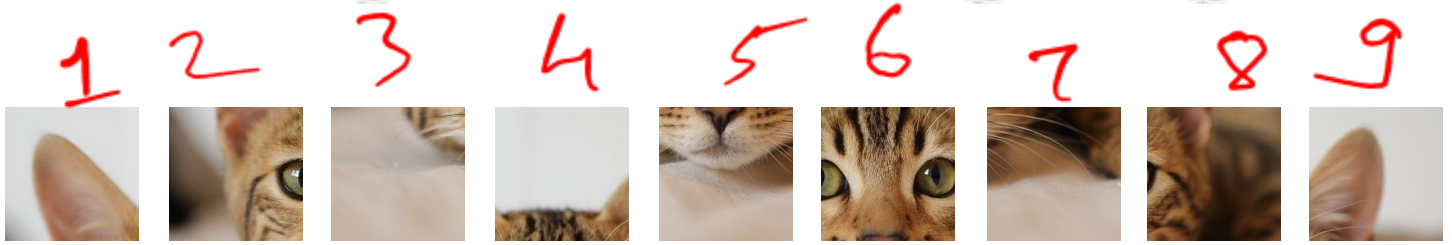
Vision Transformer (ViT)



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



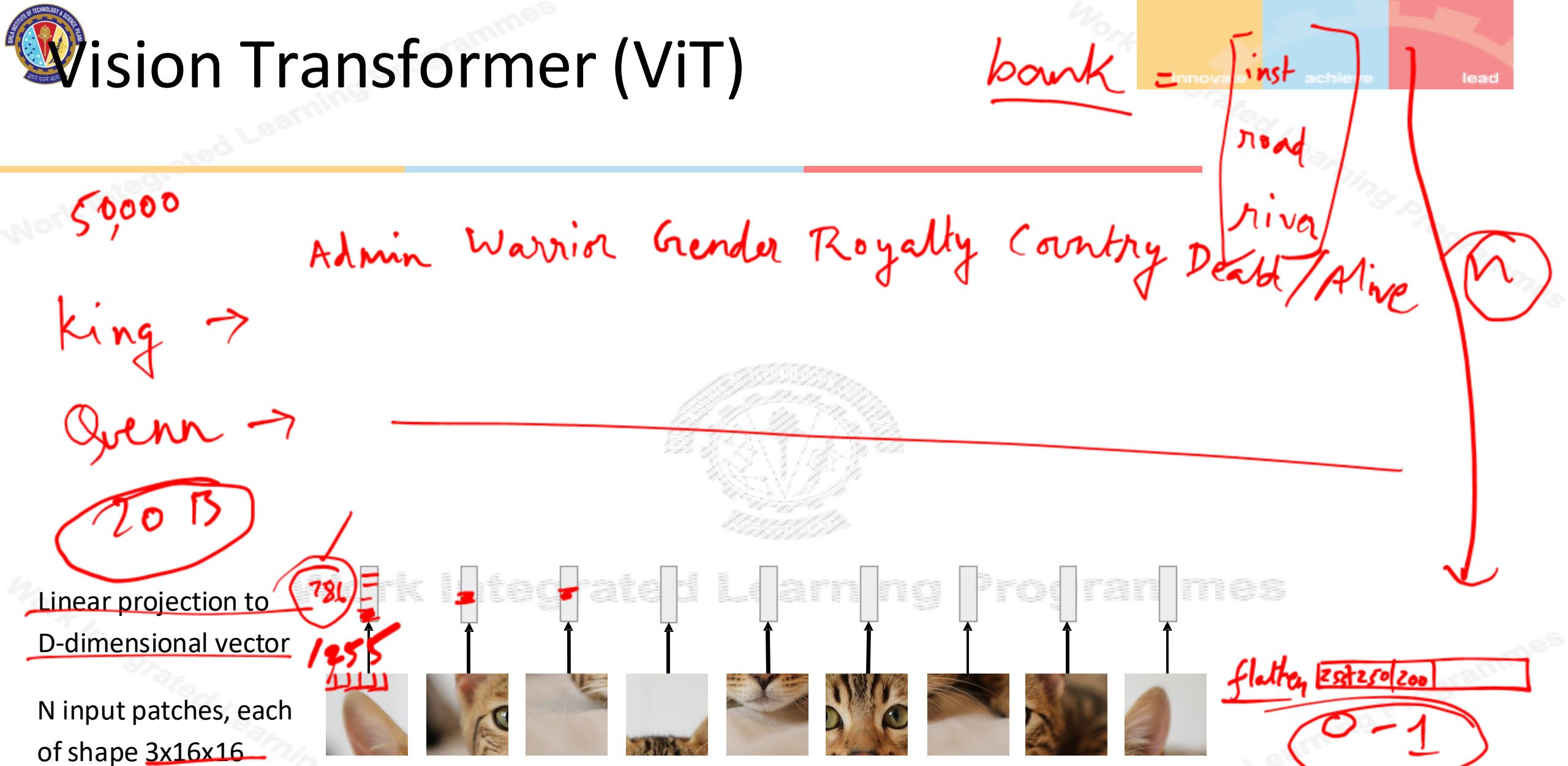
N input patches, each of shape 3x16x16

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

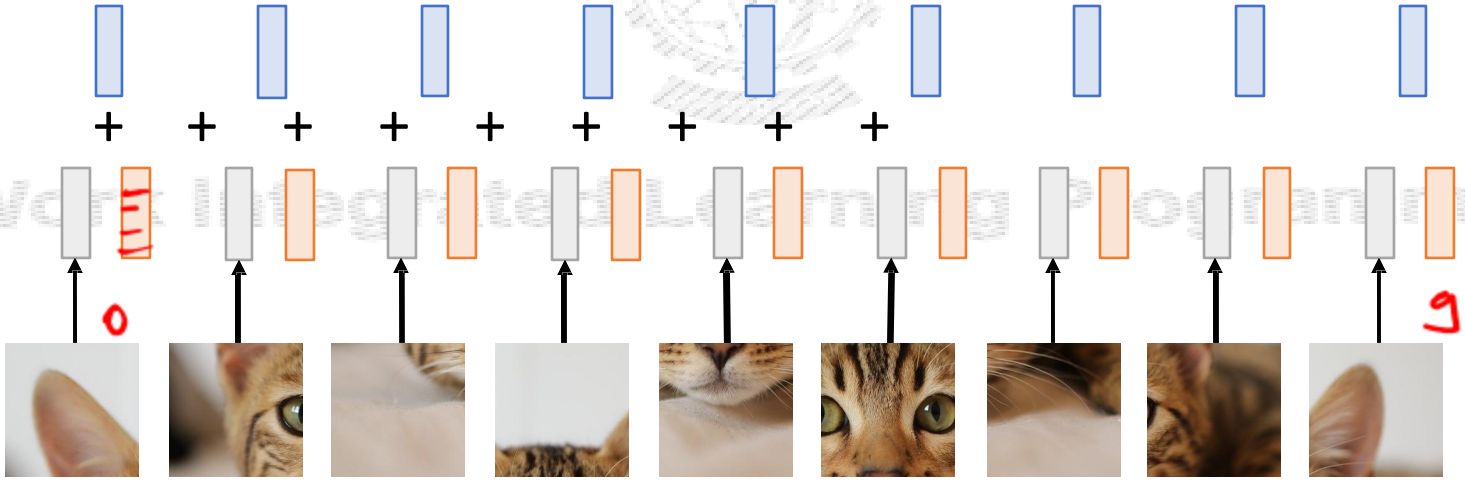
Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16

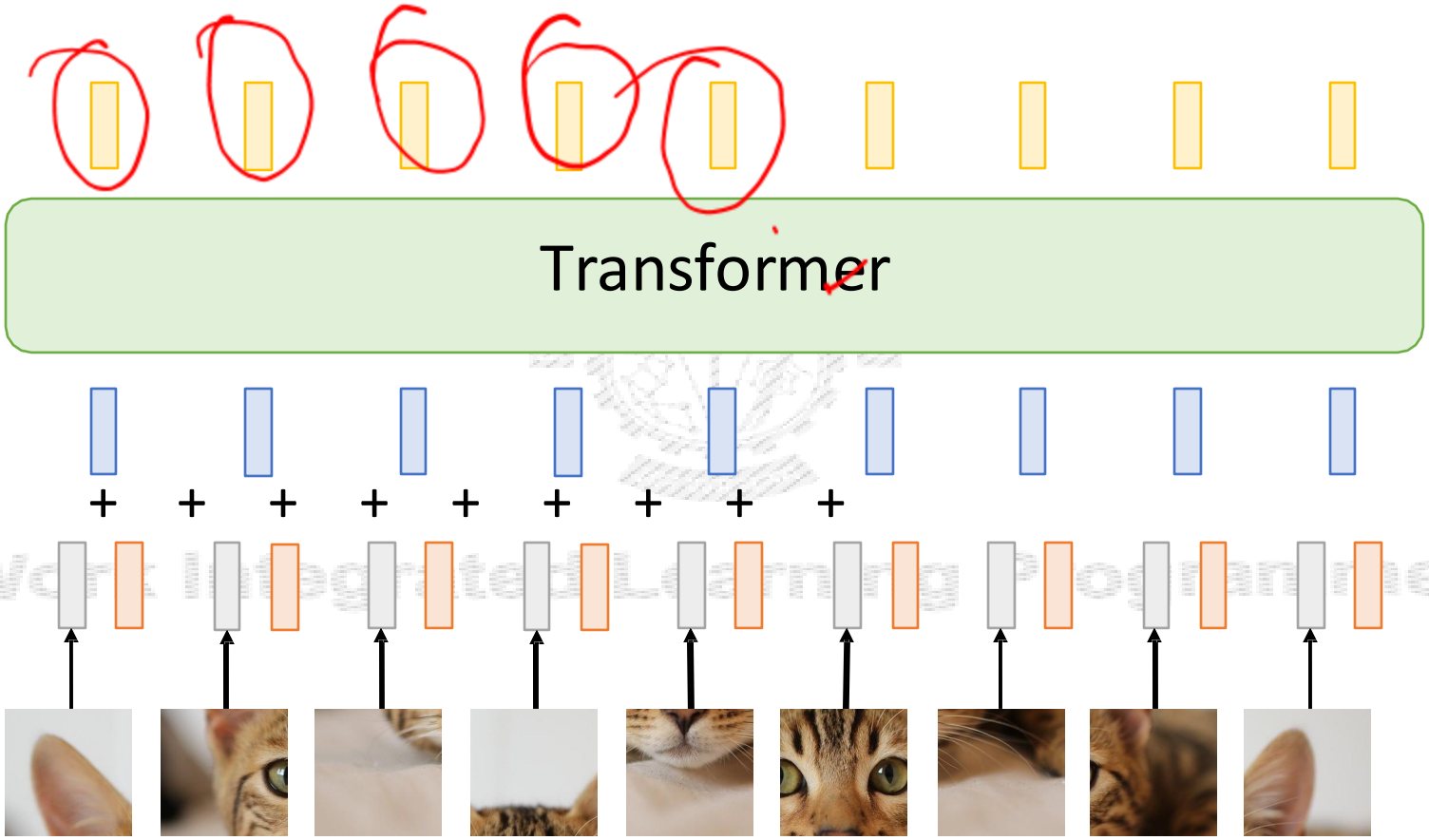


Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



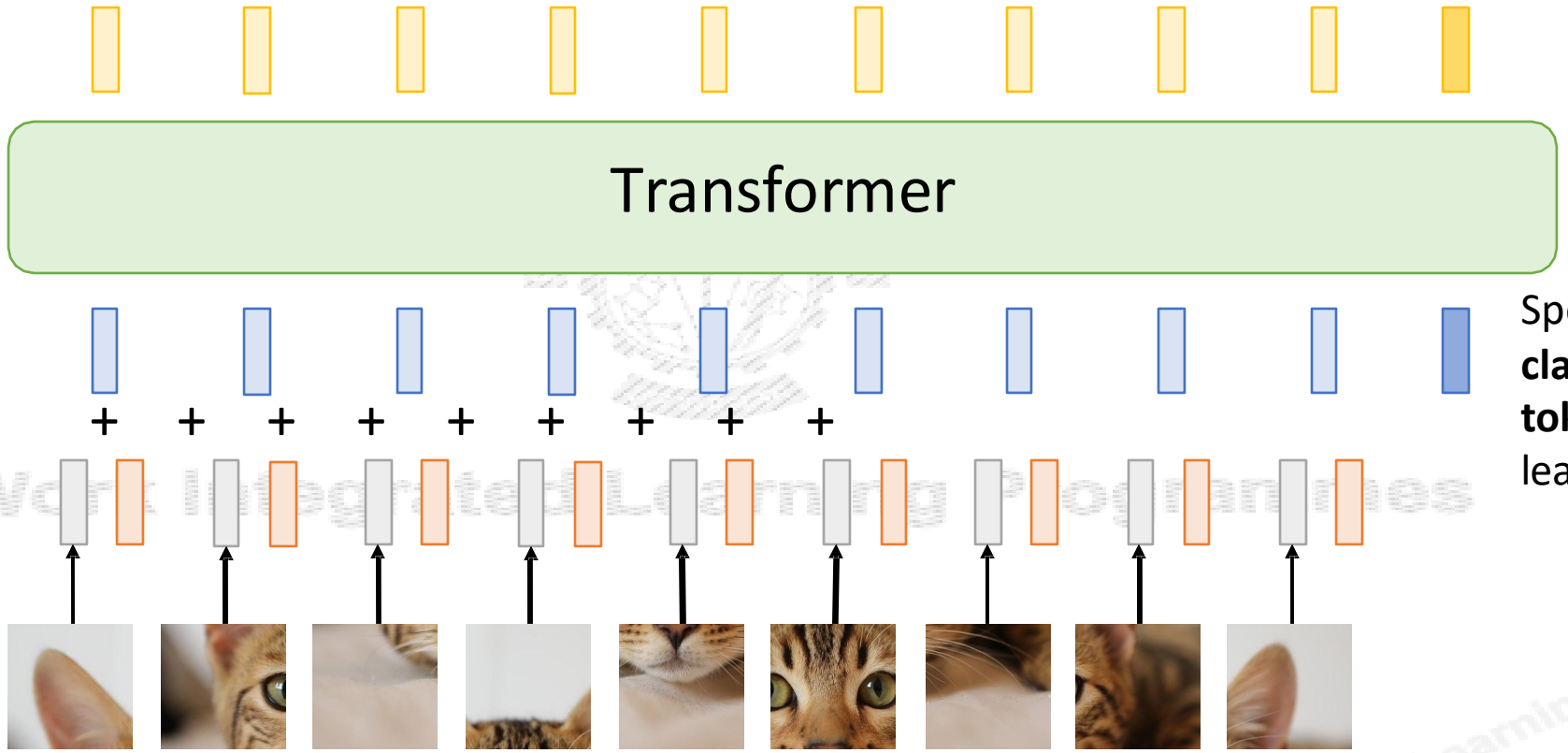
Exact same as NLP Transformer!

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



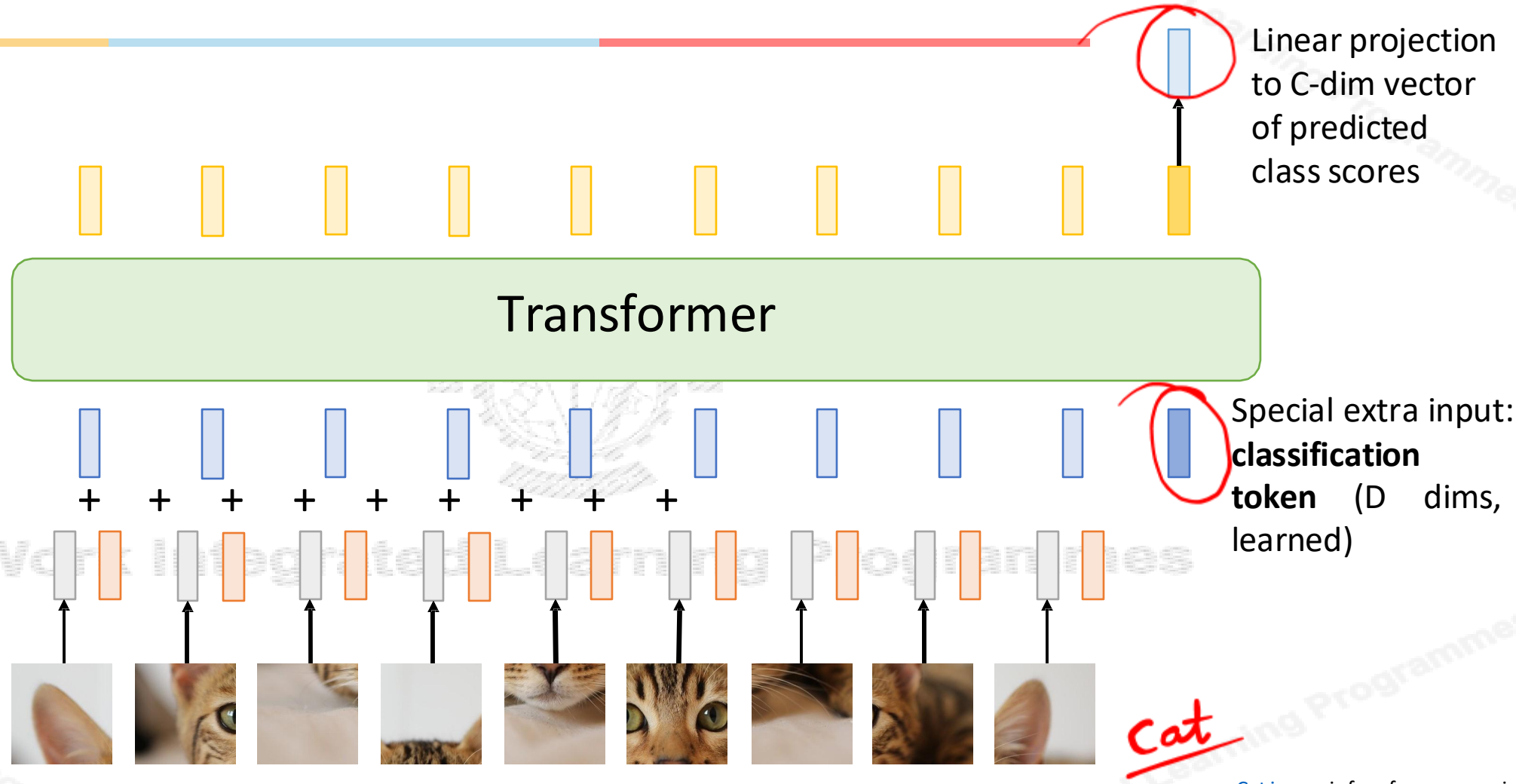
Exact same as NLP Transformer!

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Computer vision model with no convolutions!

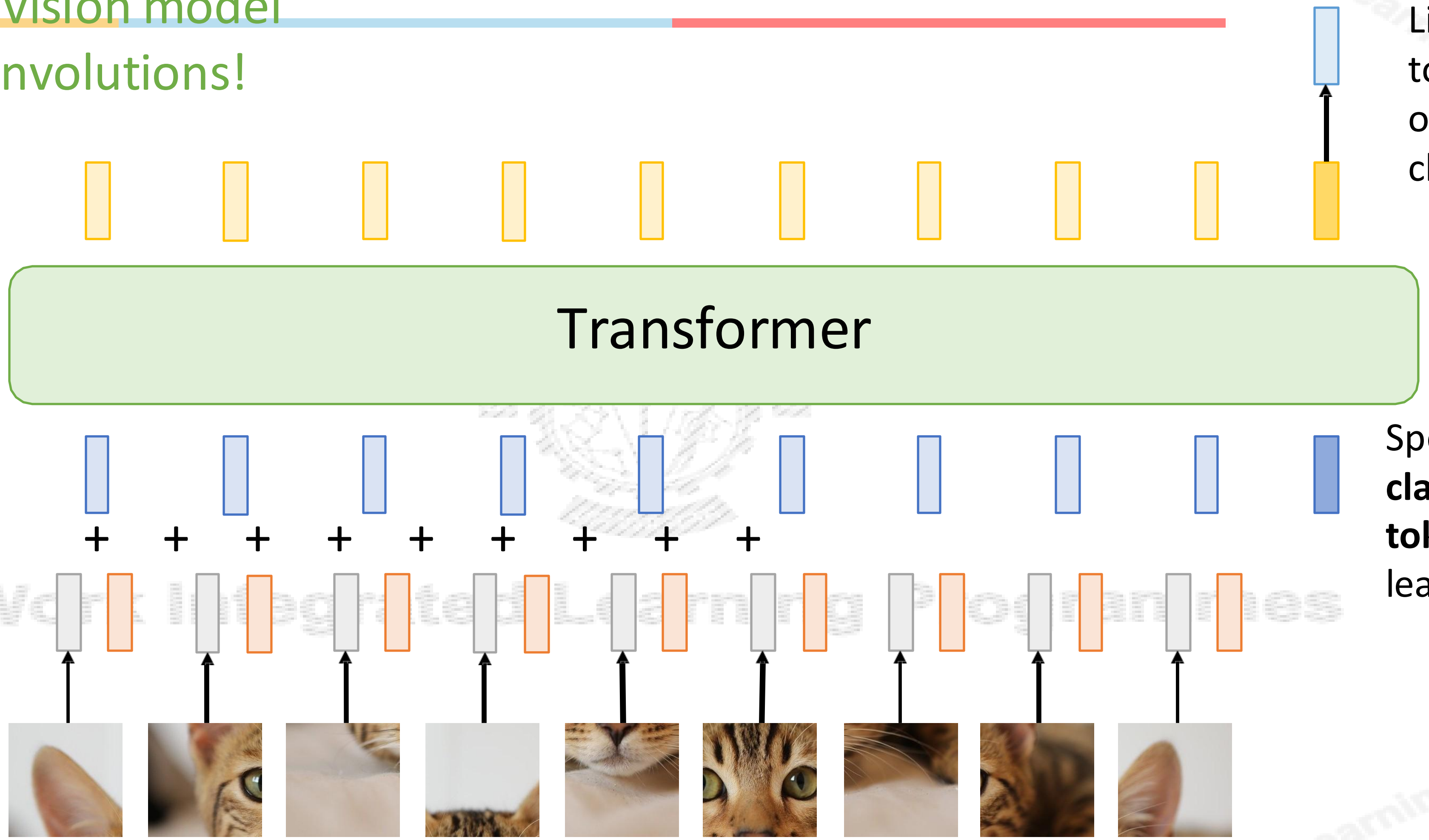
Output vectors

Exact same as NLP Transformer!

Add positional embedding: learned D-dim vector per position

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Linear projection to C-dim vector of predicted class scores

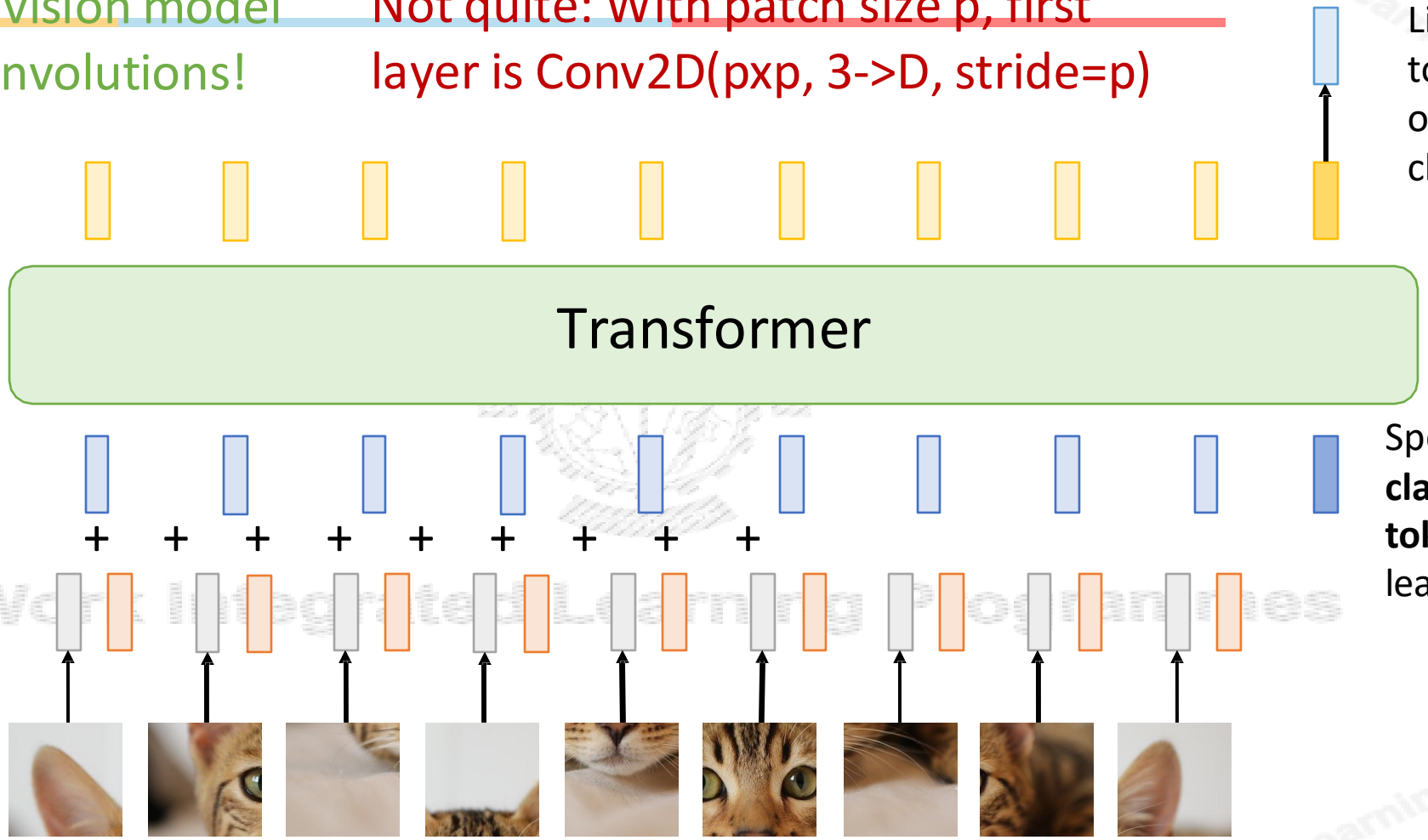
Special extra input: **classification token** (D dims, learned)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Vision Transformer (ViT)

Computer vision model with no convolutions!

Not quite: With patch size p, first layer is Conv2D(pxp, 3->D, stride=p)



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

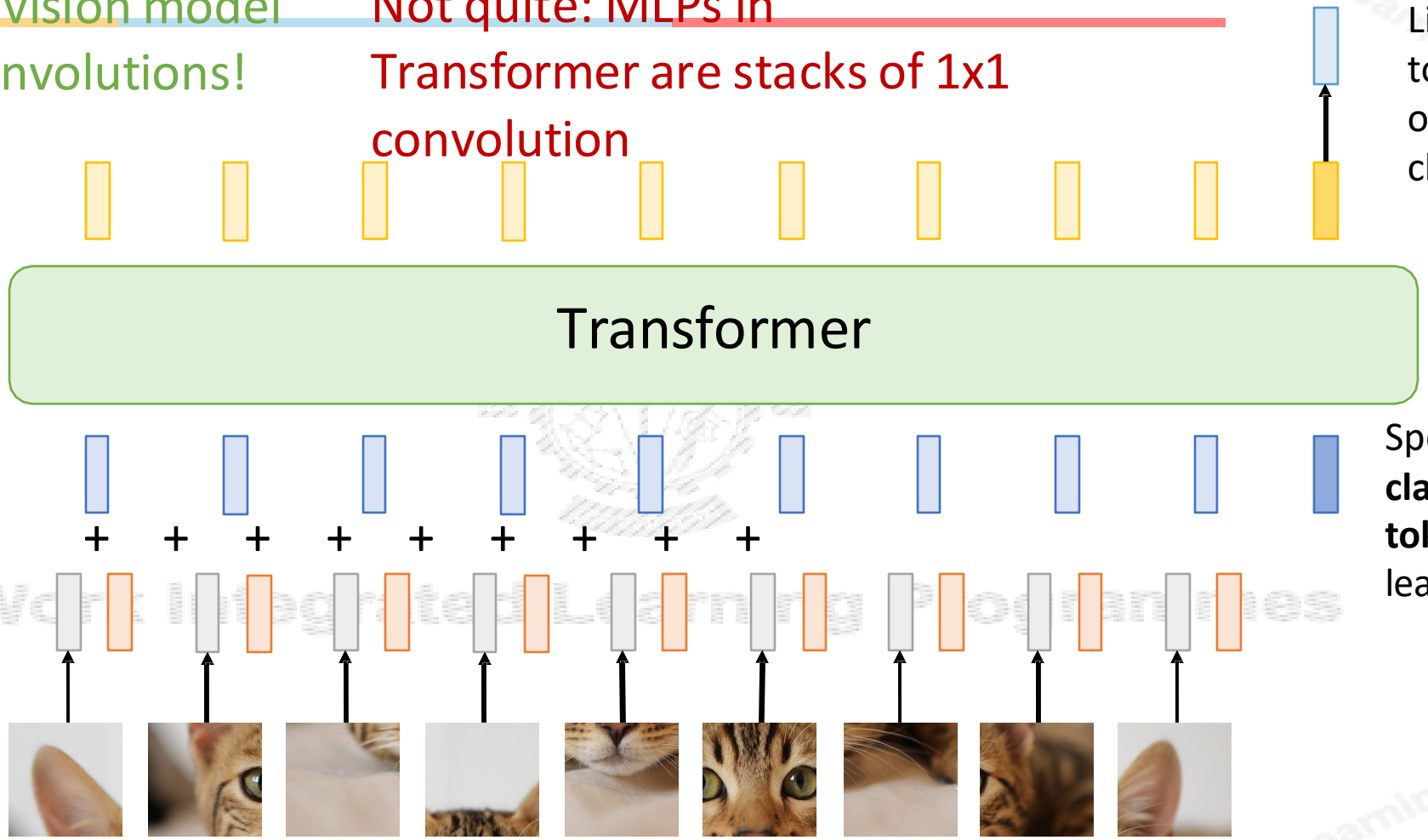
Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

Computer vision model with no convolutions!

Not quite: MLPs in Transformer are stacks of 1x1 convolution



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

Each attention matrix has $14^4 = 38,416$ entries, takes 150 KB (or 65,536 entries, takes 256 KB)

Linear projection to C-dim vector of predicted class scores

Output vectors

Exact same as NLP Transformer!

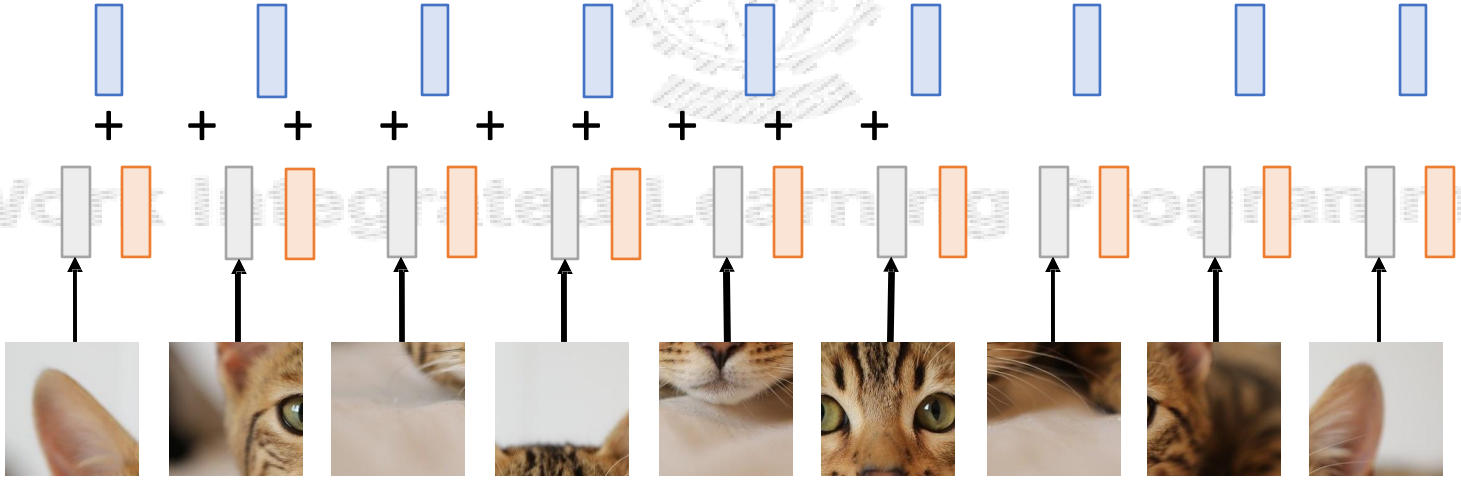
Transformer

Add positional embedding: learned D-dim vector per position

Special extra input: **classification token** (D dims, learned)

Linear projection to D-dimensional vector

N input patches, each of shape 3x16x16



Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

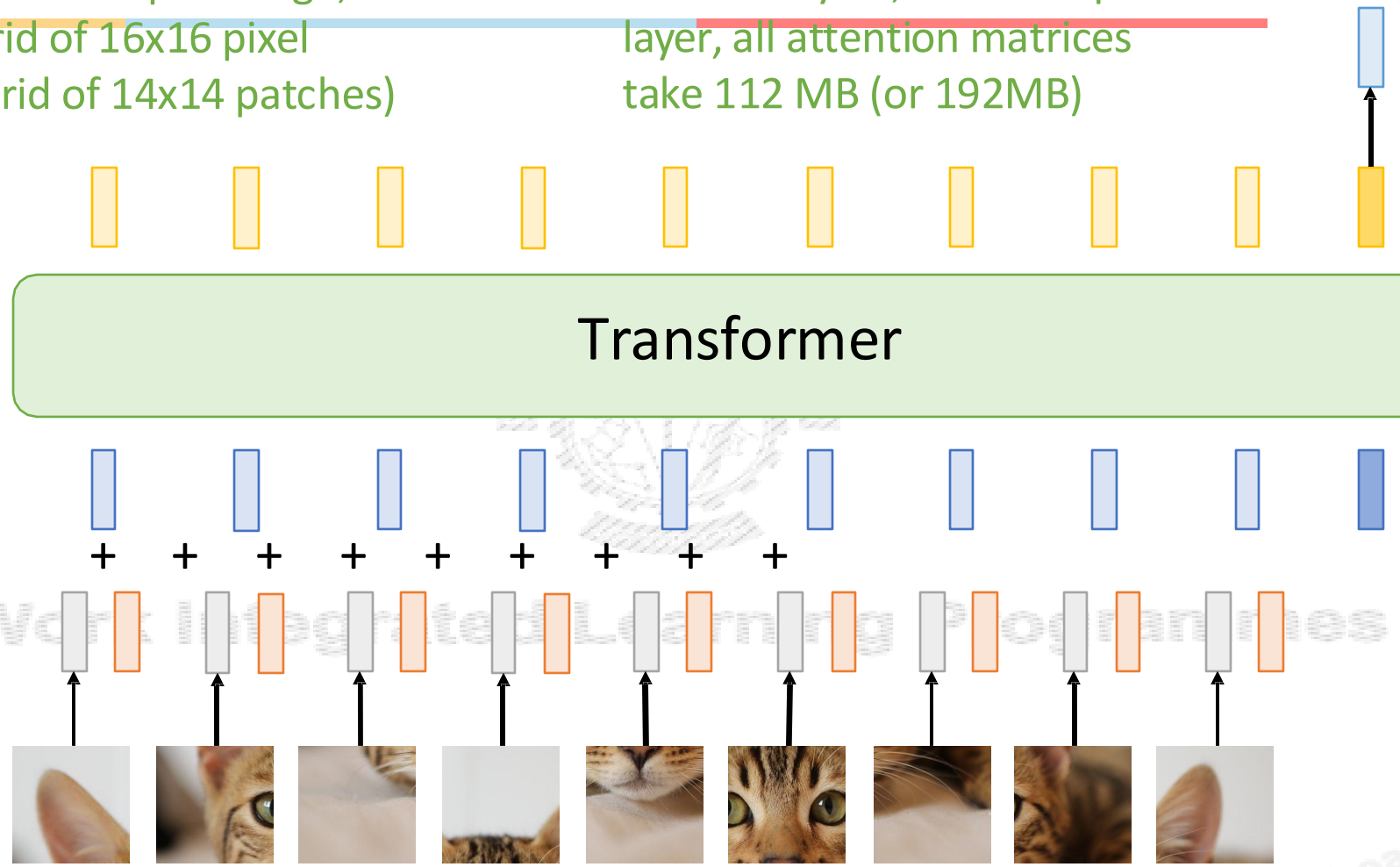
Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

With 48 layers, 16 heads per layer, all attention matrices take 112 MB (or 192MB)



Exact same as NLP Transformer!

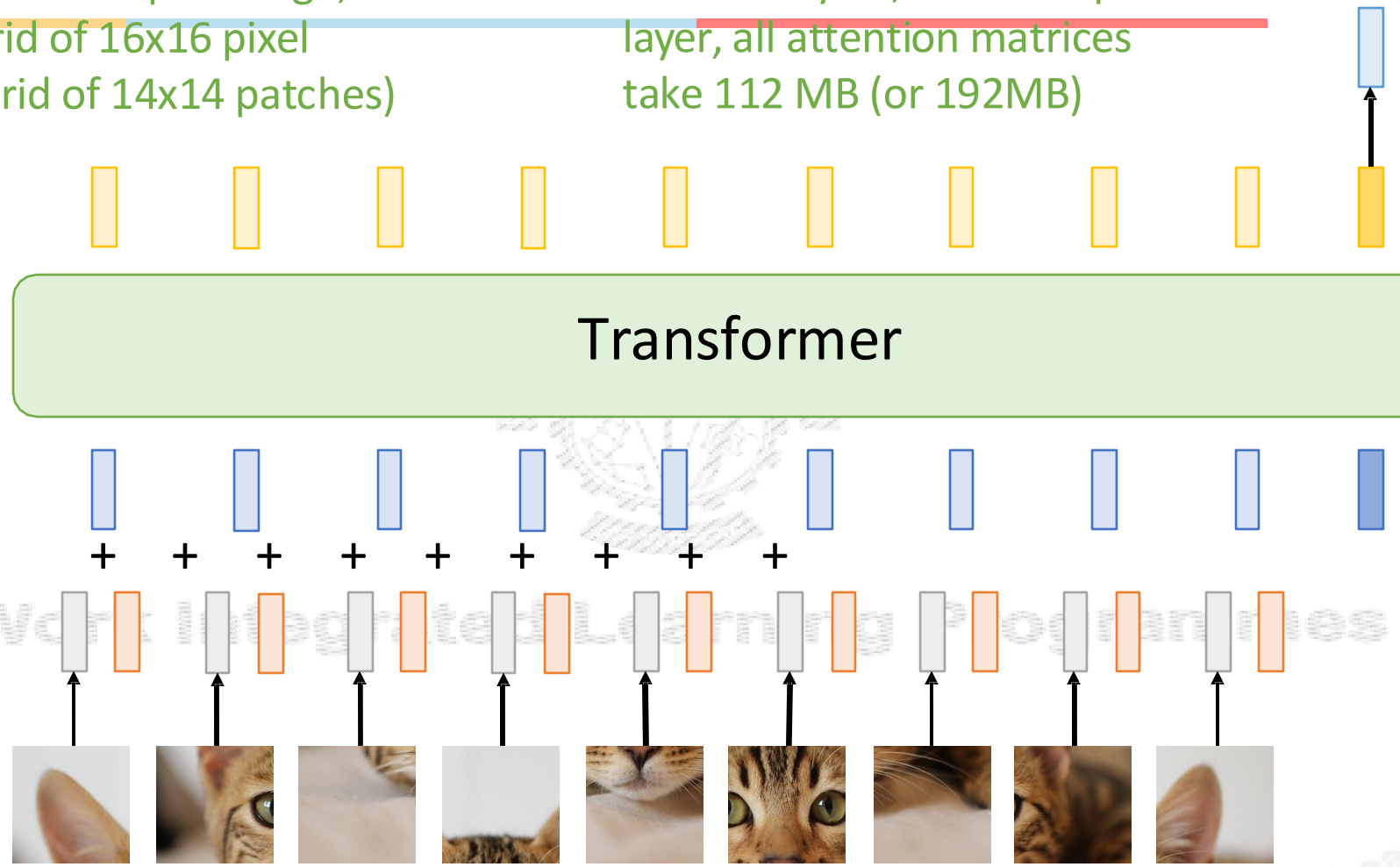
Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT)

In practice: take 224x224 input image, divide into 14x14 grid of 16x16 pixel patches (or 16x16 grid of 14x14 patches)

With 48 layers, 16 heads per layer, all attention matrices take 112 MB (or 192MB)

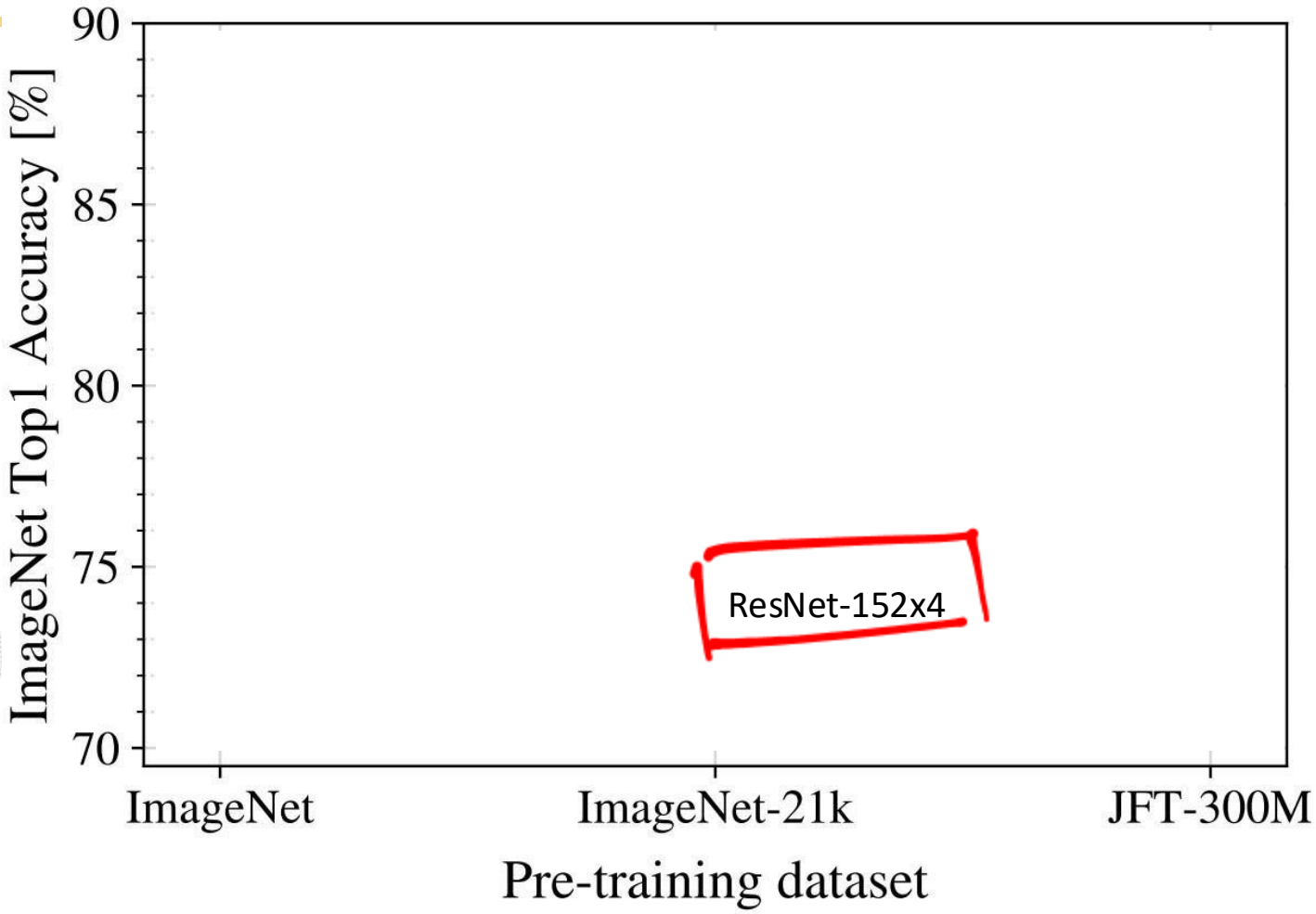


Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Cat image is free for commercial use under a Pixabay license

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

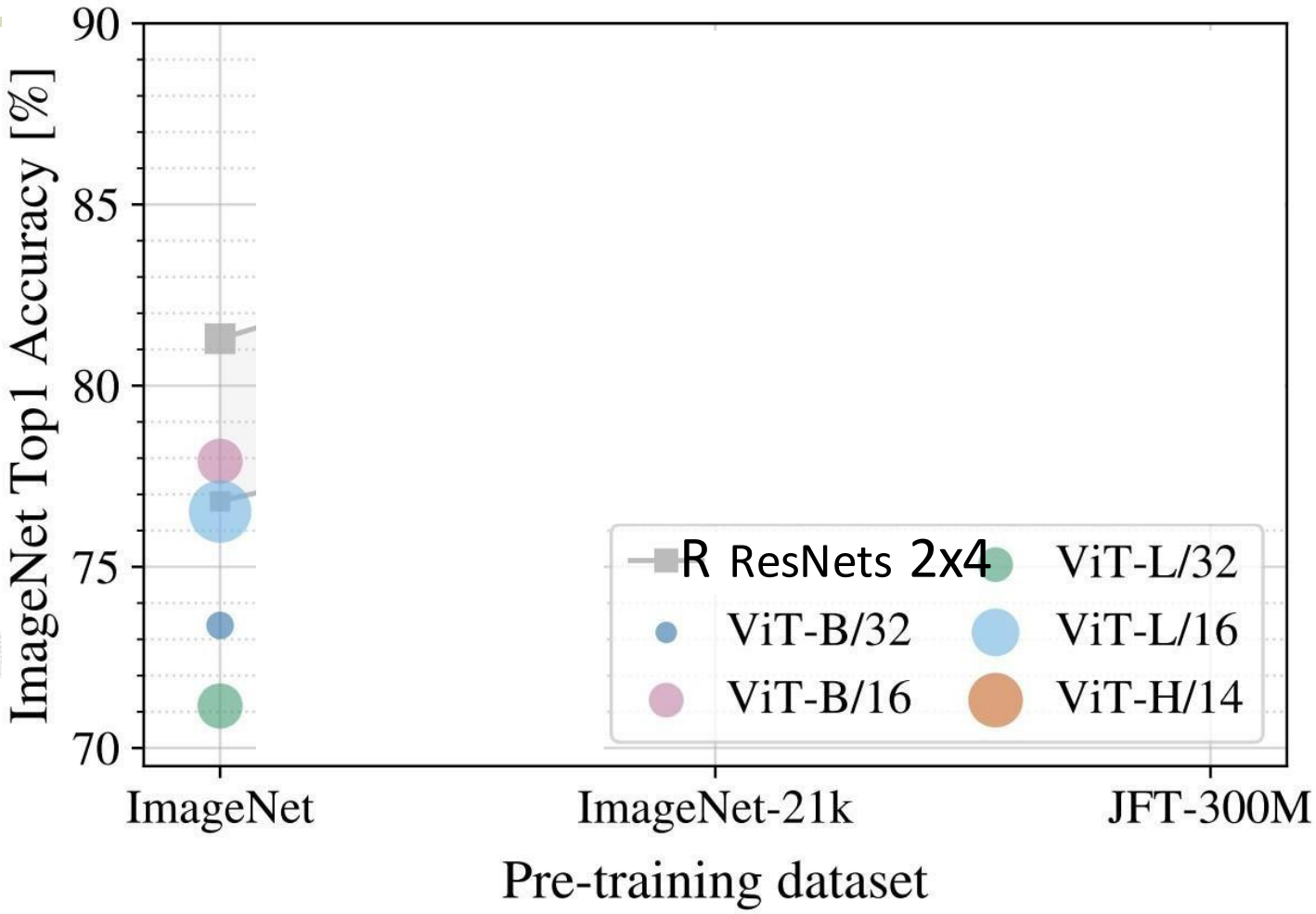
Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

Recall: ImageNet dataset has 1k categories, 1.2M images

When trained on ImageNet, ViT models perform worse than ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

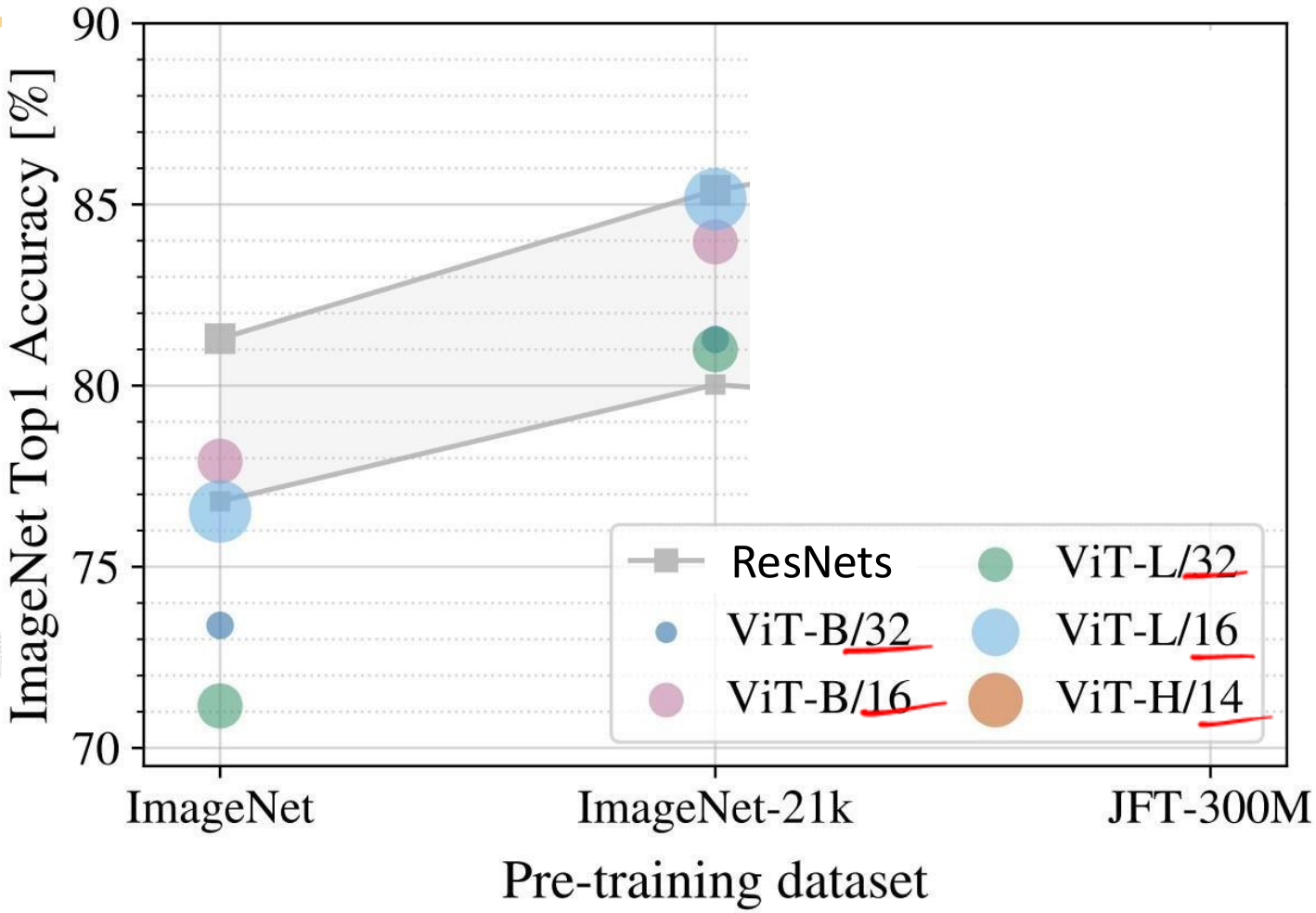
Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

ImageNet-21k has 14M images with 21k categories

If you pretrain on ImageNet-21k and fine-tune on ImageNet, ViT does better: big ViTs match big ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

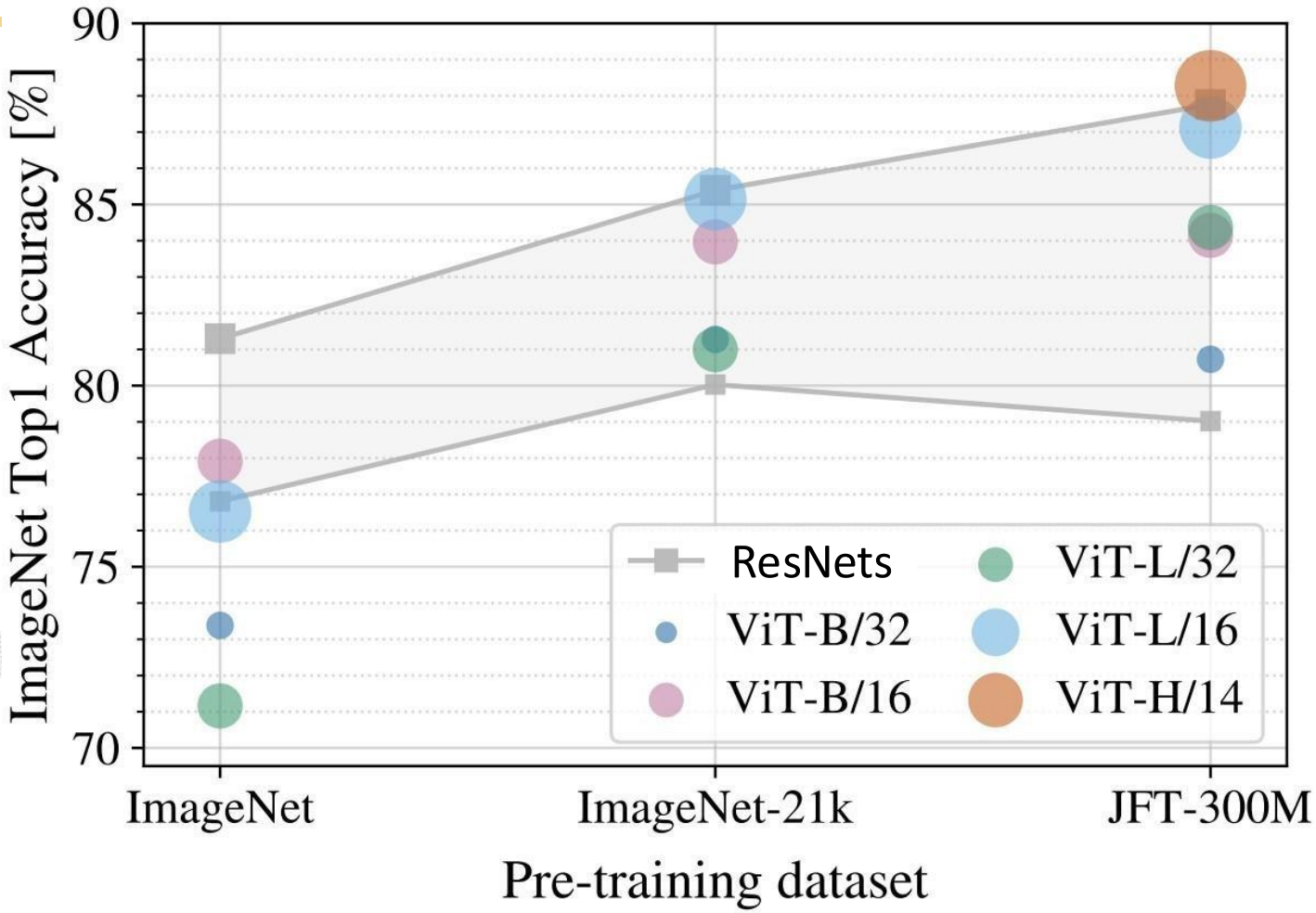
Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

JFT-300M is an internal Google dataset with 300M labeled images

If you pretrain on JFT and finetune on ImageNet, large ViTs outperform large ResNets



B = Base
L = Large
H = Huge

/32, /16, /14 is patch size; smaller patch size is a bigger model (more patches)

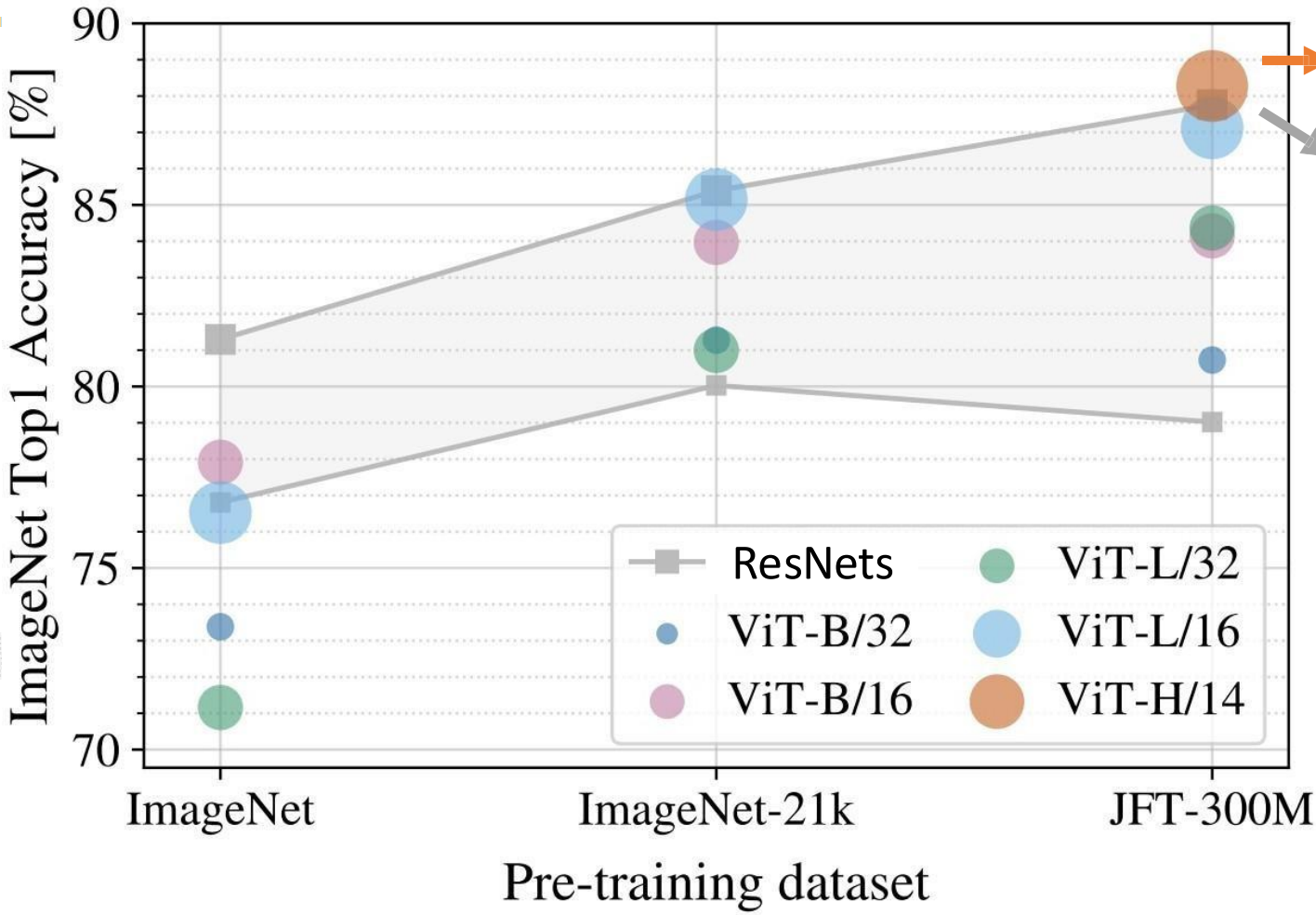
Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

JFT-300M is an internal Google dataset with 300M labeled images

If you pretrain on JFT and finetune on ImageNet, large ViTs outperform large ResNets



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

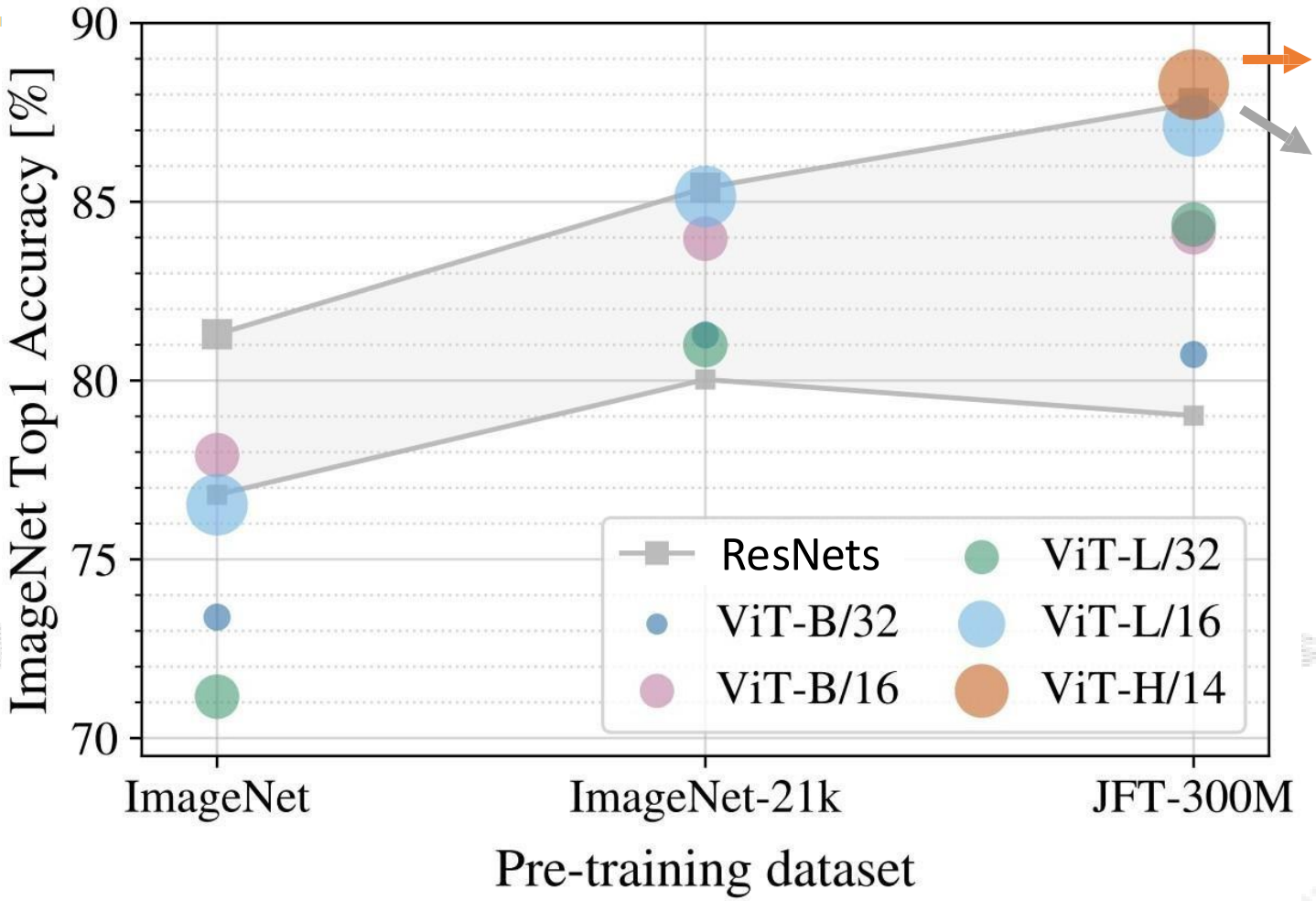
ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

Claim: ViT models have “less inductive bias” than ResNets, so need more pretraining data to learn good features
(Not sure I buy this explanation: “inductive bias” is not a well-defined concept we can measure!)



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

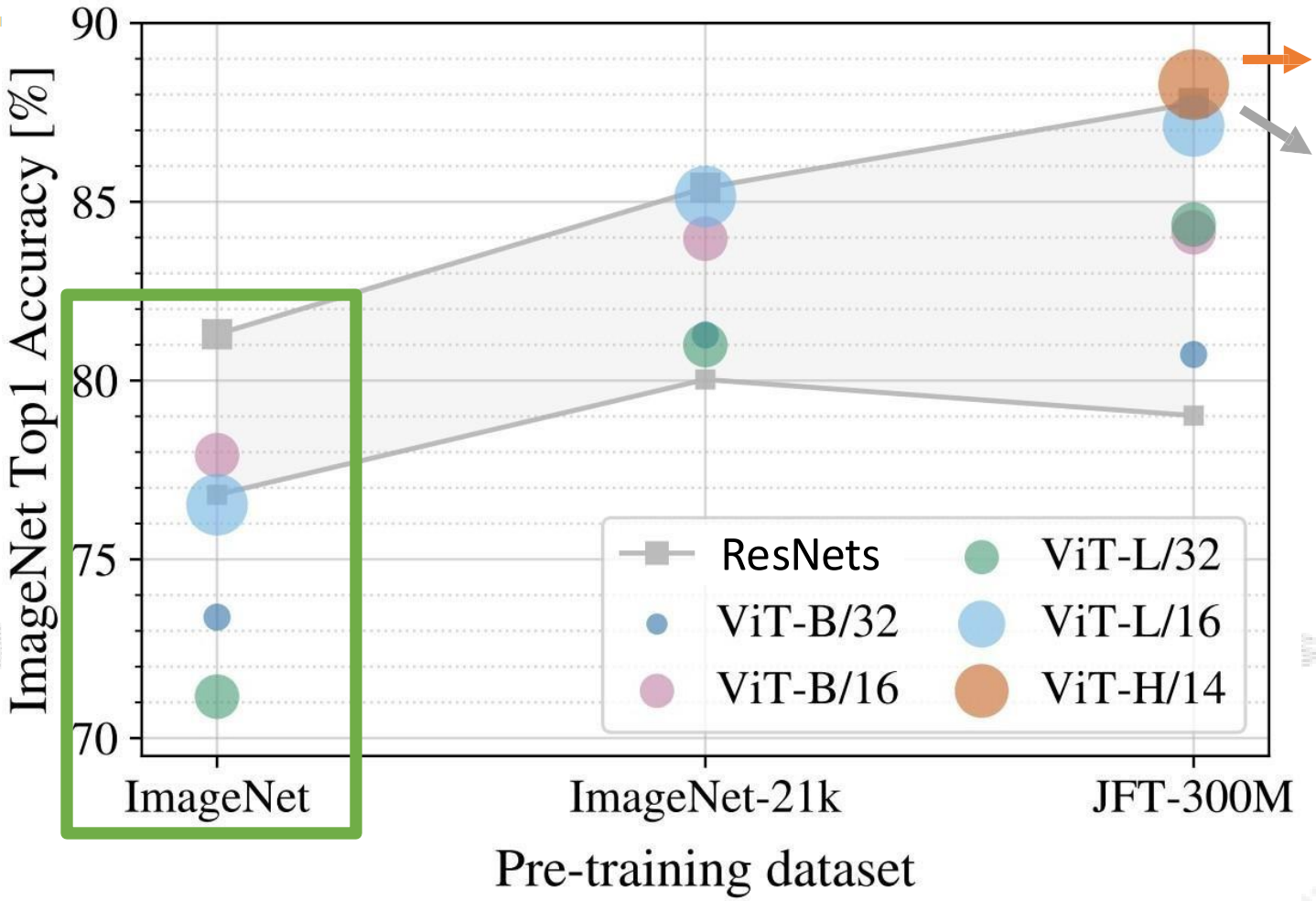
ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”, ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich

Vision Transformer (ViT) vs ResNets

How can we improve the performance of ViT models on ImageNet?



ViT: 2.5k TPU-v3 core days of training

ResNet: 9.9k TPU-v3 core days of training

ViTs make more efficient use of GPU / TPU hardware (matrix multiply is more hardware-friendly than conv)

Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ICLR 2021

Source: Slides from Justin Johnson; Course : EECS498; UMich